



Wholly owned by UTAR Education Foundation
(Co. No. 578227-M)
DU012(A)

UNIVERSITI TUNKU ABDUL RAHMAN (UTAR)

FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY (FICT)

UCCN3004 SERVER MANAGEMENT AND ADMINISTRATION

YEAR AND SEMESTER: 2024/06

Mini Project

Lecturer's Name: **Dr. Fityanul Akhyar**

Name	Programme	Student ID	Marks
FOO KAR YENG	CN	2300422	
TAN YI FEI	CN	2200842	
WONG PEI KEI	CN	2207466	

Table of Contents

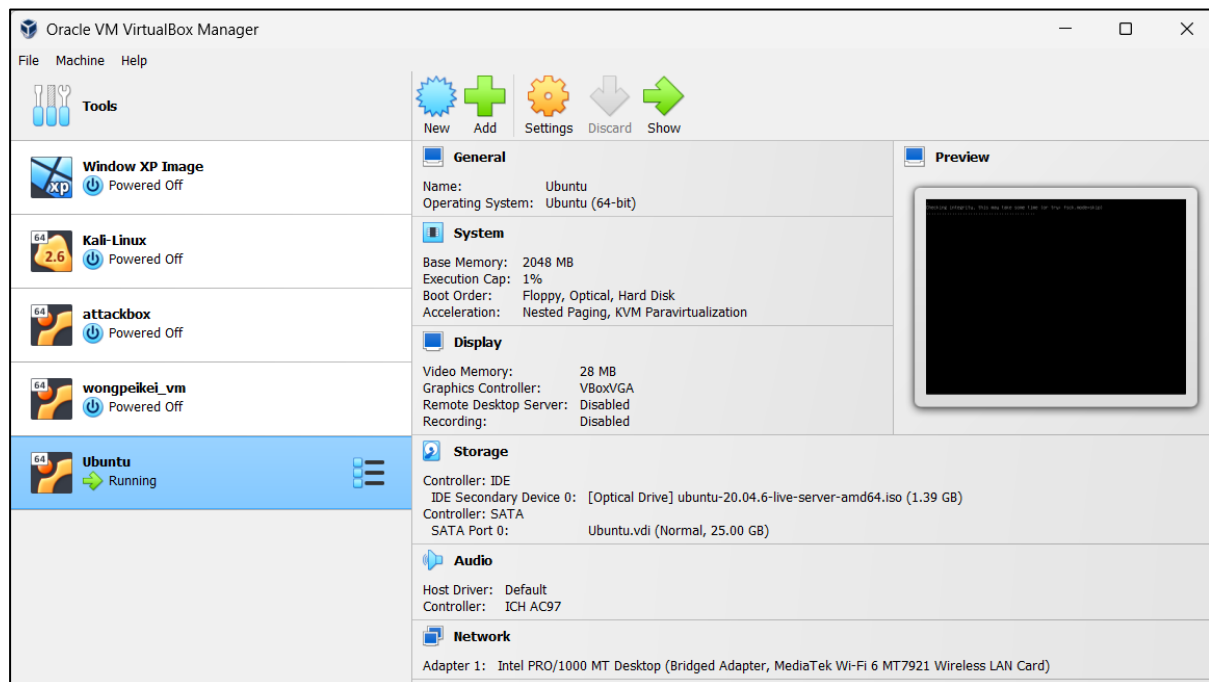
No.	Content	Page Number
1	SQUID	
	1.1 Installation	1 – 4
	1.2 Configuration	5 – 11
	1.3 System Testing	11 – 18
2	NGINX	
	2.1 Installation	19 – 20
	2.2 Configuration	21 – 30
	2.3 System Testing	31 – 35
3	HAProxy	
	3.1 Installation	36 – 37
	3.2 Configuration	38 – 45
	3.3 System Testing	46 – 50

1. SQUID Proxy

1.1 Squid Installation

1. Open Ubuntu Server in VM

Start virtual machine and ensure it's running Ubuntu Server version 20.04.6. Make sure VM is connected to the internet.



```
ubuntu login: wongpeikei
Password:
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.4.0-193-generic x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/advantage

System information as of Tue 27 Aug 2024
System information as of Tue 27 Aug 2024 11:07:41 AM UTC

System load:  0.13           Processes:            96
Usage of /:   39.2% of 11.21GB Users logged in:             0
Memory usage: 8%            IPv4 address for enp0s3: 192.168.68.142
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status
```

2. Update Ubuntu System

Before installing any new software, it's essential to update the package list and upgrade the installed packages to the latest versions.

```
wongpeikey@ubuntu:~$ sudo apt update && sudo apt upgrade -y
[sudo] password for wongpeikey:
Hit:1 http://my.archive.ubuntu.com/ubuntu focal InRelease
Hit:2 http://my.archive.ubuntu.com/ubuntu focal-updates InRelease
Hit:3 http://my.archive.ubuntu.com/ubuntu focal-backports InRelease
Hit:4 http://my.archive.ubuntu.com/ubuntu focal-security InRelease
Reading package lists... 47%
```

3. Install Squid Proxy

Install Squid, a caching proxy for the web that supports HTTP, HTTPS, and FTP. The -y flag automatically confirms the installation.

```
wongpeikey@ubuntu:~$ sudo apt-get install squid -y
Reading package lists... Done
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
```

4. Verify Installation

After installation, verify that Squid has been installed correctly by checking its version.

```
wongpeikey@ubuntu:~$ squid -v
Squid Cache: Version 4.10
Service Name: squid
Ubuntu linux
configure options: '--build=x86_64-linux-gnu' '--prefix=/usr' '--includedir=${prefix}/include' '--m
andir=${prefix}/share/man' '--infodir=${prefix}/share/info' '--sysconfdir=/etc' '--localstatedir=/va
r' '--libexecdir=${prefix}/lib/squid' '--srcdir=' '--disable-maintainer-mode' '--disable-dependency
-tracking' '--disable-silent-rules' 'BUILDCXXFLAGS=-g -O2 -fdebug-prefix-map=/build/squid-moVrwM/squ
id-4.10=. -fstack-protector-strong -Wformat -Werror=format-security -std=c++17 -Wdate-time -D_FORTIF
Y_SOURCE=2 -Wl,-Bsymbolic-functions -Wl,-z,relro -Wl,-z,now -Wl,--as-needed' 'BUILDCXX=x86_64-linux-
gnu-g++' '--with-build-environment=default' '--enable-build-info=Ubuntu linux' '--datadir=/usr/share
/squid' '--sysconfdir=/etc/squid' '--libexecdir=/usr/lib/squid' '--mandir=/usr/share/man' '--enable-
inline' '--disable-arch-native' '--enable-async-io=8' '--enable-storeio=ufs,aufs,diskd,rock' '--enab
le-removal-policies=lru,heap' '--enable-delay-pools' '--enable-cache-digests' '--enable-icap-client'
'--enable-follow-x-forwarded-for' '--enable-auth-basic=DB,fake,getpwnam,LDAP,NCSA,NIS,PAM,POP3,RADI
US,SASL,SMB' '--enable-auth-digest=file,LDAP' '--enable-auth-negotiate=kerberos,wrapper' '--enable-a
uth-ntlm=fake,SMB_LM' '--enable-external-acl-helpers=file_userip,kerberos_ldap_group,LDAP_group,sess
ion,SQL_session,time_quota,unix_group,wbinfo_group' '--enable-security-cert-validators=fake' '--enab
le-storeid-rewrite-helpers=file' '--enable-url-rewrite-helpers=fake' '--enable-eui' '--enable-esi' '
--enable-icmp' '--enable-zph-qos' '--enable-ecap' '--disable-translation' '--with-swapdir=/var/spool
/squid' '--with-logdir=/var/log/squid' '--with-pidfile=/var/run/squid.pid' '--with-filedescriptors=6
5536' '--with-large-files' '--with-default-user=proxy' '--with-gnutls' '--enable-linux-netfilter' 'b
uild_alias=x86_64-linux-gnu' 'CC=x86_64-linux-gnu-gcc' 'CFLAGS=-g -O2 -fdebug-prefix-map=/build/squ
id-moVrwM/squid-4.10=. -fstack-protector-strong -Wformat -Werror=format-security -Wall' 'LDFLAGS=-Wl,
-Bsymbolic-functions -Wl,-z,relro -Wl,-z,now -Wl,--as-needed' 'CPPFLAGS=-Wdate-time -D_FORTIFY_SOURC
E=2' 'CXX=x86_64-linux-gnu-g++' 'CXXFLAGS=-g -O2 -fdebug-prefix-map=/build/squid-moVrwM/squid-4.10=.
-fstack-protector-strong -Wformat -Werror=format-security -std=c++17'
wongpeikey@ubuntu:~$ _
```

5. Status of Squid Service

Ensure Squid is running correctly by checking its service status. It shows the current status of the Squid service, indicating whether it's active (running) or inactive (stopped).

```
wongpeikei@ubuntu:~$ sudo systemctl status squid
● squid.service - Squid Web Proxy Server
   Loaded: loaded (/lib/systemd/system/squid.service; enabled; vendor preset: enabled)
   Active: active (running) since Tue 2024-08-27 11:31:05 UTC; 13min ago
     Docs: man:squid(8)
    Main PID: 18205 (squid)
      Tasks: 4 (limit: 2256)
    Memory: 16.1M
    CGroup: /system.slice/squid.service
            └─18205 /usr/sbin/squid -sYC
               └─18207 (squid-1) --kid squid-1 -sYC
                  └─18208 (logfile-daemon) /var/log/squid/access.log
                     └─18209 (pinger)

Aug 27 11:31:05 ubuntu squid[18207]: Max Swap size: 0 KB
Aug 27 11:31:05 ubuntu squid[18207]: Using Least Load store dir selection
Aug 27 11:31:05 ubuntu squid[18207]: Set Current Directory to /var/spool/squid
Aug 27 11:31:05 ubuntu squid[18207]: Finished loading MIME types and icons.
Aug 27 11:31:05 ubuntu squid[18207]: HTCP Disabled.
Aug 27 11:31:05 ubuntu squid[18207]: Pinger socket opened on FD 14
Aug 27 11:31:05 ubuntu squid[18207]: Squid plugin modules loaded: 0
Aug 27 11:31:05 ubuntu squid[18207]: Adaptation support is off.
Aug 27 11:31:05 ubuntu squid[18207]: Accepting HTTP Socket connections at local=[::]:3128 remote=[::]:3128
Aug 27 11:31:06 ubuntu squid[18207]: storeLateRelease: released 0 objects
lines 1-23/23 (END)
```

6. Install Firewall

Install the latest firewall for enhanced security, use the command `sudo apt install ufw` for latest traffic agreements.

```
wongpeikei@ubuntu:~$ sudo apt install ufw
[sudo] password for wongpeikei:
Reading package lists... Done
Building dependency tree
Reading state information... Done
ufw is already the newest version (0.36-6ubuntu1.1).
ufw set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
```

7. Check Local IP Address

Identify the IP address of Ubuntu server, which is `192.168.68.142/24` will be used in the Squid configuration.

```
wongpeikei@ubuntu:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:dc:74:6c brd ff:ff:ff:ff:ff:ff
    inet 192.168.68.142/24 brd 192.168.68.255 scope global dynamic enp0s3
        valid_lft 2830sec preferred_lft 2830sec
    inet6 fe80::a00:27ff:fedc:746c/64 scope link
        valid_lft forever preferred_lft forever
```

1.2 Squid Configuration

Squid's configuration is controlled by the main configuration file located at `"/etc/squid/squid.conf"`.

1. Backup Original Configuration File

Before making any changes, it's crucial to back up the original configuration file. This command creates a backup of the Squid configuration file, ensures to restore it if needed.

```
wongpeikei@ubuntu:~$ sudo cp /etc/squid/squid.conf /etc/squid/squid.conf.backup
```

2. Edit Squid Configuration File

Open the Squid configuration file for editing. In nano file, use `ctrl+w` to search within the file. Search for specific sections like `"http_access"` and `"http_port"` to make necessary changes.

```
wongpeikei@ubuntu:~$ sudo nano /etc/squid/squid.conf
[sudo] password for wongpeikei:
```

3. Allow Squid Traffic

Configure the Uncomplicated Firewall (UFW) to allow traffic on Squid's default port (3128). These commands install UFW, allow TCP traffic on port 3128, and check the current firewall status to confirm the changes.

```
wongpeikei@ubuntu:~$ sudo ufw allow 3128/tcp
Rules updated
Rules updated (v6)
wongpeikei@ubuntu:~$
```

```
wongpeikei@ubuntu:~$ sudo ufw status
Status: active

To Action From
--
3128/tcp ALLOW Anywhere
3128 ALLOW Anywhere
3128/tcp (v6) ALLOW Anywhere (v6)
3128 (v6) ALLOW Anywhere (v6)
```

4. Configure HTTP Port

Change the HTTP port to bind Squid to my server's IP address `192.168.68.142` and listen on port `3128`.

```
# Squid normally listens to port 3128
http_port 192.168.68.142:3128
```

5. Set Up Cache Directory

Specify the cache directory where Squid will store cached objects. This configuration sets up a 100MB cache size using the UFS (Unix File System) in the `/var/spool/squid` directory. The 16 256 parameters define the number of first-level and second-level directories.

```
# Uncomment and adjust the following to add a disk cache directory.
cache_dir ufs /var/spool/squid 100 16 256
```

Use this command to ensure the cache directory is initialized.

```
wongpeikei@ubuntu:~$ sudo squid -z
2024/08/31 00:38:46| FATAL: Squid is already running: Found fresh instance PID file (/var/run/squid.
pid) with PID 25692
exception location: Instance.cc(121) ThrowIfAlreadyRunningWith
```

6. Allow Access from Local Network

Configure Squid to allow access from your local network and deny all other access. This configuration creates an access control list (ACL) for your local network and allows access only from this network and localhost. All other access is denied.

```
# Allow Access from Local Network
acl localnet src 192.168.68.0/24
http_access allow localnet

# Example rule allowing access from your local networks.
# Adapt localnet in the ACL section to list your (internal) IP networks
# from where browsing should be allowed

# Allow Access from localhost
http_access allow localhost

# And finally deny all other access to this proxy
http_access deny all
```

7. Enable Access Logging

Enable logging of all access requests for monitoring purposes. This command logs access requests to the specified log file.

```
#      Default:
#      access_log daemon:/var/log/squid/access.log squid
#Default:
access_log /var/log/squid/access.log squid
```

8. Set Up Squid Authentication

Configure basic authentication for Squid using Apache2 utilities. These commands install the necessary utilities, create a password file, and add a user (squid) with a password.

```
wongpeikey@ubuntu:~$ sudo apt install apache2-utils -y
Reading package lists... Done
Building dependency tree
Reading state information... Done
apache2-utils is already the newest version (2.4.41-4ubuntu3.21).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
```

```
wongpeikey@ubuntu:~$ sudo touch /etc/squid/password
wongpeikey@ubuntu:~$ sudo chown proxy: /etc/squid/password
wongpeikey@ubuntu:~$ sudo htpasswd /etc/squid/password squid
New password:
Re-type new password:
Adding password for user squid
```

```
wongpeikey@ubuntu:~$ sudo cat /etc/squid/password
squid:$apr1$HsWbspq1$Dbm8i2SAQfgbYSKkDHyLQ1
wongpeikey@ubuntu:~$
```

Configure Squid to use the authentication. This configuration sets up basic authentication using the credentials stored in */etc/squid/password*.

```
# Private Connection
auth_param basic program /usr/lib/squid3/basic_ncsa_auth /etc/squid/password
auth_param basic realm proxy
acl authenticated proxy_auth REQUIRED
http_access allow authenticated

# Allow Access from Local Network
acl localnet src 192.168.68.0/24
http_access allow localnet

# Example rule allowing access from your local networks.
# Adapt localnet in the ACL section to list your (internal) IP networks
# from where browsing should be allowed

# Allow Access from localhost
http_access allow localhost

# And finally deny all other access to this proxy
http_access deny all
```

9. Blocking Websites with Squid Proxy

Configure Squid to define rules to block access to specific websites. Add the following lines to block sites like Facebook, Instagram, and Twitter.

```
# Block Access on Domain
acl blocksites dstdomain .facebook.com .instagram.com .twitter.com
http_access deny blocksites

# Private Connection
auth_param basic program /usr/lib/squid3/basic_ncsa_auth /etc/squid/password
auth_param basic realm proxy
acl authenticated proxy_auth REQUIRED
http_access allow authenticated

# Allow Access from Local Network
acl localnet src 192.168.68.0/24
http_access allow localnet

# Example rule allowing access from your local networks.
# Adapt localnet in the ACL section to list your (internal) IP networks
# from where browsing should be allowed

# Allow Access from localhost
http_access allow localhost

# And finally deny all other access to this proxy
http_access deny all
```

10. Block Downloads

Prevent the download of specific file types like .exe, .zip, and .mp3. This squid configuration blocks access to URLs that match the specified file extensions.

```
# Block Download
acl blockfiles urlpath_regex -i \.exe$ \.zip$ \.mp3$
http_access deny blockfiles

# Block Access on Domain
acl blocksites dstdomain .facebook.com .instagram.com .twitter.com
http_access deny blocksites

# Private Connection
auth_param basic program /usr/lib/squid3/basic_ncsa_auth /etc/squid/password
auth_param basic realm proxy
acl authenticated proxy_auth REQUIRED
http_access allow authenticated

# Allow Access from Local Network
acl localnet src 192.168.68.0/24
http_access allow localnet

# Example rule allowing access from your local networks.
# Adapt localnet in the ACL section to list your (internal) IP networks
# from where browsing should be allowed

# Allow Access from localhost
http_access allow localhost

# And finally deny all other access to this proxy
http_access deny all
```

11. Set Work Hours

Restrict access based on specific work hours. This configuration denies access outside the specified time range 10AM-7PM on weekdays (Monday to Friday).

```
include /etc/squid/conf.d/*  
  
# Block Access on Work Hours  
acl workhours time MTWTF 10:00-19:00  
http_access deny workhours  
  
# Block Download  
acl blockfiles urlpath_regex -i \.exe$ \.zip$ \.mp3$  
http_access deny blockfiles  
  
# Block Access on Domain  
acl blocksites dstdomain .facebook.com .instagram.com .twitter.com  
http_access deny blocksites  
  
# Private Connection  
auth_param basic program /usr/lib/squid3/basic_ncsa_auth /etc/squid/password  
auth_param basic realm proxy  
acl authenticated proxy_auth REQUIRED  
http_access allow authenticated  
  
# Allow Access from Local Network  
acl localnet src 192.168.68.0/24
```

12. Restrict Squid to Specific Ports

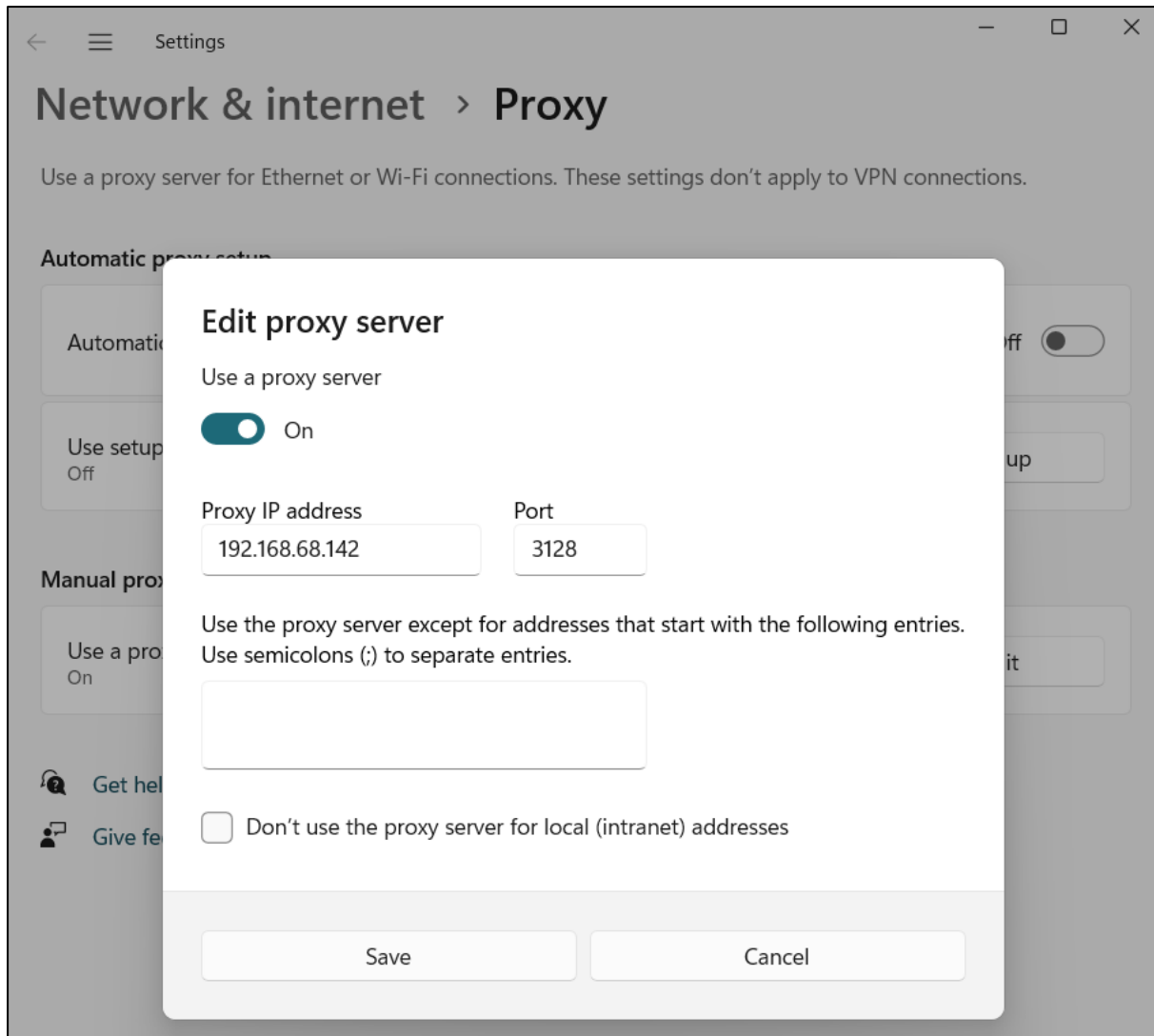
The configuration restricts Squid to only allow traffic on specific ports. This setup ensures that Squid only permits traffic on ports 80, 443, and 21, and blocks access to any other ports to enhance security.

```
include /etc/squid/conf.d/*  
  
# Restrict to Specific Ports  
acl Safe_ports port 80  
acl Safe_ports port 443  
acl Safe_ports port 21  
acl CONNECT method CONNECT  
http_access deny !Safe_ports  
  
# Block Access on Work Hours  
acl workhours time MTWTF 10:00-19:00  
http_access deny workhours
```

13. Configuring Squid Proxy on Client Machine

Set up my client machines to use the Squid proxy server by configuring their proxy settings with the server's IP address and port:

- IP address: 192.168.68.142
- Port: 3128



14. Restart Squid Service

Restart the Squid service to apply all the changes made to the configuration file. This command restarts the Squid service, ensuring that the new settings take effect.

```
wongpeikei@ubuntu:~$ sudo systemctl restart squid
```

```
wongpeikei@ubuntu:~$ sudo systemctl enable squid
Synchronizing state of squid.service with SysV service script with /lib/systemd/systemd-sysv-install
.
Executing: /lib/systemd/systemd-sysv-install enable squid
wongpeikei@ubuntu:~$ _
```

```
wongpeikei@ubuntu:~$ sudo systemctl status squid
• squid.service - Squid Web Proxy Server
   Loaded: loaded (/lib/systemd/system/squid.service; enabled; vendor preset: enabled)
   Active: active (running) since Sat 2024-08-31 09:54:00 +08; 15s ago
     Docs: man:squid(8)
  Process: 27462 ExecStartPre=/usr/sbin/squid --foreground -z (code=exited, status=0/SUCCESS)
  Process: 27466 ExecStart=/usr/sbin/squid -sYC (code=exited, status=0/SUCCESS)
 Main PID: 27467 (squid)
    Tasks: 4 (limit: 2256)
   Memory: 16.6M
   CGroup: /system.slice/squid.service
           └─27467 /usr/sbin/squid -sYC
             └─27469 (squid-1) --kid squid-1 -sYC
               └─27470 (unlinkd)
                 └─27471 (pinger)

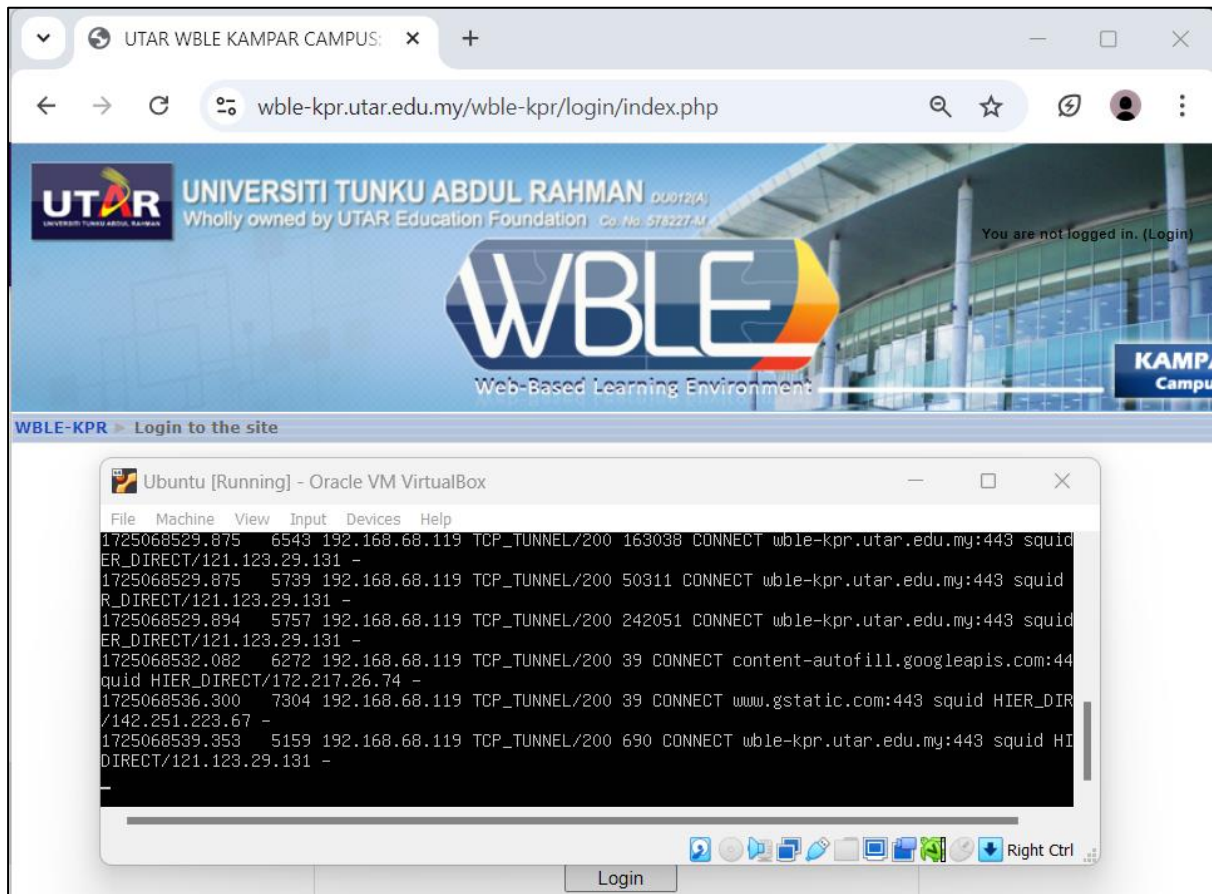
Aug 31 09:54:01 ubuntu squid[27469]:      0 Objects expired.
Aug 31 09:54:01 ubuntu squid[27469]:      0 Objects cancelled.
Aug 31 09:54:01 ubuntu squid[27469]:      0 Duplicate URLs purged.
Aug 31 09:54:01 ubuntu squid[27469]:      0 Swapfile clashes avoided.
Aug 31 09:54:01 ubuntu squid[27469]: Took 0.01 seconds (205.28 objects/sec).
Aug 31 09:54:01 ubuntu squid[27469]: Beginning Validation Procedure
Aug 31 09:54:01 ubuntu squid[27469]: Completed Validation Procedure
Aug 31 09:54:01 ubuntu squid[27469]: Validated 3 Entries
Aug 31 09:54:01 ubuntu squid[27469]: store_swap_size = 12.00 KB
Aug 31 09:54:02 ubuntu squid[27469]: storeLateRelease: released 0 objects
```

1.3 System Testing

1. Test Squid Access Logs

Monitor the access log to verify that requests are being logged. This command displays real-time logs of requests processed by Squid.

```
wongpeikei@ubuntu:~$ sudo tail -f /var/log/squid/access.log
```



2. Test Squid Connectivity using Curl

Install curl, a command-line tool to transfer data from or to a server. Use curl to test the Squid proxy by making HTTP requests.

```
wongpeikei@ubuntu:~$ sudo apt install curl
Reading package lists... Done
Building dependency tree
Reading state information... Done
curl is already the newest version (7.68.0-1ubuntu2.23).
curl set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
```

3. Test Squid Caching Behavior

Test the caching behavior of Squid with two sequential curl requests:

1st Request (Cache Miss) - X-Cache-Lookup: MISS indicates that the request was not found in the cache and was fetched from the origin server.

```
wongpei@ubuntu:~$ sudo systemctl restart squid
wongpei@ubuntu:~$ curl -x http://192.168.68.142:3128 --proxy-user squid:squid -I -L http://utar.edu.my
HTTP/1.1 301 Moved Permanently
Date: Sat, 31 Aug 2024 07:04:28 GMT
Server: Apache
Location: https://utar.edu.my/
Content-Type: text/html; charset=iso-8859-1
X-Cache: MISS from ubuntu
X-Cache-Lookup: MISS from ubuntu:3128
Via: 1.1 ubuntu (squid/4.10)
Connection: keep-alive

HTTP/1.1 200 Connection established

HTTP/1.1 200 OK
Date: Sat, 31 Aug 2024 07:04:29 GMT
Server: Apache
Content-Type: text/html; charset=UTF-8
```

2nd Request (Cache Hit) - Result: X-Cache-Lookup: HIT indicates that the request was served from the cache.

```
wongpei@ubuntu:~$ curl -x http://192.168.68.142:3128 --proxy-user squid:squid -I -L http://utar.edu.my
HTTP/1.1 301 Moved Permanently
Date: Sat, 31 Aug 2024 07:05:22 GMT
Server: Apache
Location: https://utar.edu.my/
Content-Type: text/html; charset=iso-8859-1
X-Cache: MISS from ubuntu
X-Cache-Lookup: HIT from ubuntu:3128
Via: 1.1 ubuntu (squid/4.10)
Connection: keep-alive

HTTP/1.1 200 Connection established

HTTP/1.1 200 OK
Date: Sat, 31 Aug 2024 07:05:24 GMT
Server: Apache
Content-Type: text/html; charset=UTF-8
```

4. Test Proxy Authentication

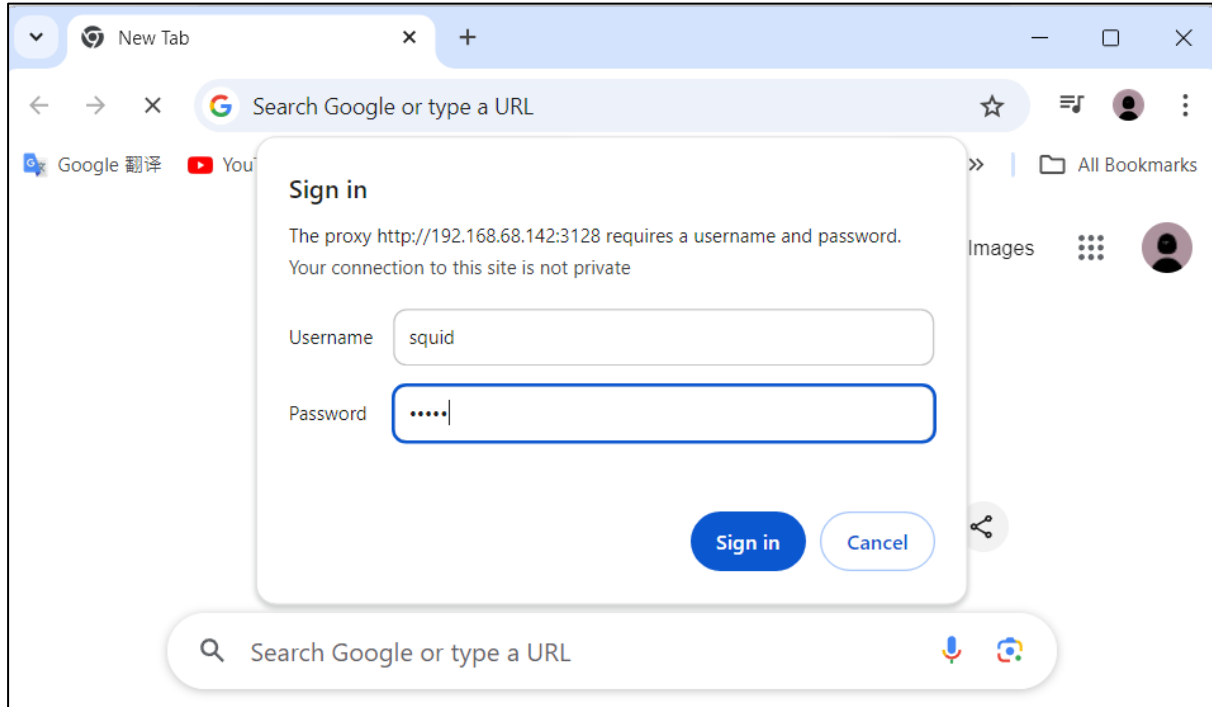
Test access with and without proxy authentication:

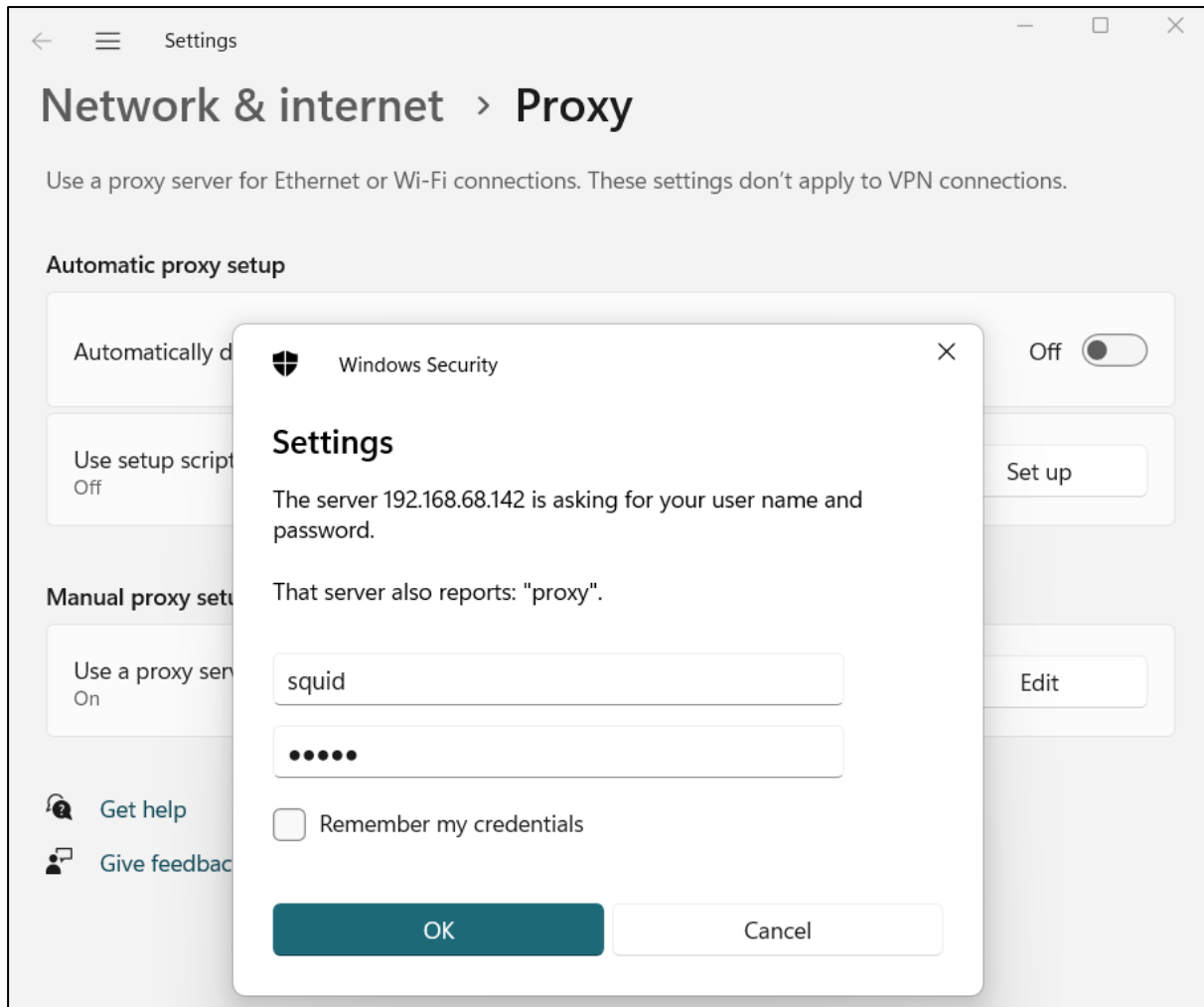
With correct credentials - Result: HTTP/1.1 301 Moved Permanently indicates successful access.

```
wongpeikei@ubuntu:~$ curl -x http://192.168.68.142:3128 --proxy-user squid:squid -I http://utar.edu.my
HTTP/1.1 301 Moved Permanently
Date: Fri, 30 Aug 2024 18:28:33 GMT
Server: Apache
Location: https://utar.edu.my/
Content-Type: text/html; charset=iso-8859-1
X-Cache: MISS from ubuntu
X-Cache-Lookup: HIT from ubuntu:3128
Via: 1.1 ubuntu (squid/4.10)
Connection: keep-alive
```

Without credentials: Result: HTTP/1.1 407 Proxy Authentication Required indicates access is blocked due to missing authentication.

```
wongpeikei@ubuntu:~$ curl -x http://192.168.68.142:3128 -I http://utar.edu.my
HTTP/1.1 407 Proxy Authentication Required
Server: squid/4.10
Mime-Version: 1.0
Date: Fri, 30 Aug 2024 18:24:31 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 3513
X-Squid-Error: ERR_CACHE_ACCESS_DENIED 0
Vary: Accept-Language
Content-Language: en
Proxy-Authenticate: Basic realm="proxy"
X-Cache: MISS from ubuntu
X-Cache-Lookup: NONE from ubuntu:3128
Via: 1.1 ubuntu (squid/4.10)
Connection: keep-alive
```





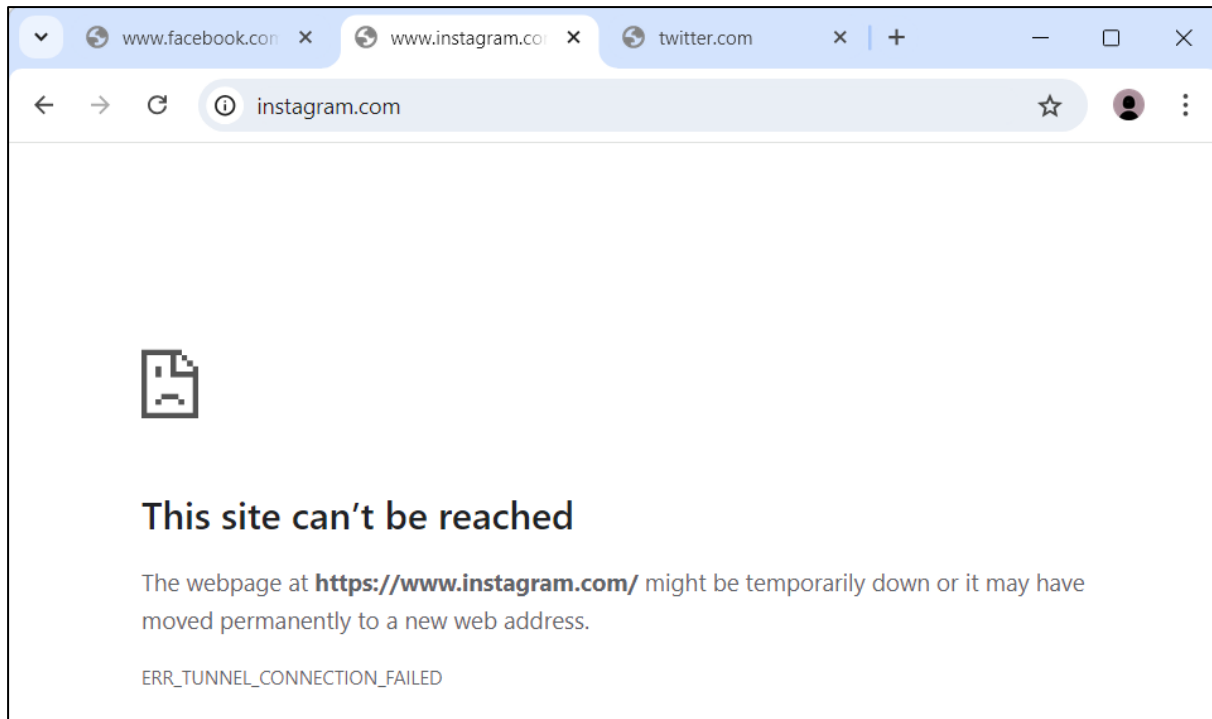
5. Verify Website Blocking

Test access to blocked websites:

Result: HTTP/1.1 403 Forbidden indicates access is denied.

Verify this in a web browser; blocked websites like Facebook, Instagram, and Twitter should be inaccessible.

```
wongpei@ubuntu:~$ curl -x http://192.168.68.142:3128 --proxy-user squid:squid -I http://facebook.com
HTTP/1.1 403 Forbidden
Server: Squid/4.10
Mime-Version: 1.0
Date: Fri, 30 Aug 2024 18:44:15 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 3560
X-Squid-Error: ERR_ACCESS_DENIED 0
Vary: Accept-Language
Content-Language: en
X-Cache: MISS from ubuntu
X-Cache-Lookup: NONE from ubuntu:3128
Via: 1.1 ubuntu (squid/4.10)
Connection: keep-alive
```



6. Test File Download Restrictions

Test blocking of file downloads:

Blocked file type: Result: HTTP/1.1 403 Forbidden indicates the file download is blocked.

```
lunneneikei@ubuntu:~$ curl -I -x http://192.168.68.142:3128 http://7-zip.org/a/7z2408-x64.exe
HTTP/1.1 403 Forbidden
Server: Squid/4.10
Mime-Version: 1.0
Date: Sat, 31 Aug 2024 02:03:59 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 3544
X-Squid-Error: ERR_ACCESS_DENIED 0
Vary: Accept-Language
Content-Language: en
X-Cache: MISS from ubuntu
X-Cache-Lookup: NONE from ubuntu:3128
Via: 1.1 ubuntu (squid/4.10)
Connection: keep-alive
```

Allowed file type: Result: HTTP/1.1 200 Connection established indicates the file download is allowed.

```
wongpeikey@ubuntu:~$ curl -I -x http://192.168.68.142:3128 --proxy-user squid:squid -I https://filesamples.com/samples/document/text/sample1.txt
HTTP/1.1 200 Connection established
HTTP/2 200
date: Sat, 31 Aug 2024 02:07:19 GMT
content-type: text/plain
cache-control: public, max-age=2592000
display: staticcontent_sol
etag: W/"64a5bc02-25f-gzip"
last-modified: Wed, 05 Jul 2023 18:52:50 GMT
response: 200
vary: Accept-Encoding,Origin
x-ezoic-cdn: Bypass
x-middleton-display: staticcontent_sol
x-middleton-response: 200
cf-cache-status: MISS
report-to: {"endpoints":[{"url":"https://a.ne1.cloudflare.com/report/v4?s=XjNR00JlnJH%2BnfgjVf92bXk7f60o2tKJ2DgVoM6KPIGfNFxcQ7uLBWHeOpCB5Af69aVrcrcyGzBtiHrmH6u3uYpMCgTg1znRS69PJGKIzc59CeGFhuGK1omTXHAXG%2Bq%2BRmVE%3D"}],"group":"cf-nel","max_age":604800}
nel: {"success_fraction":0,"report_to":"cf-nel","max_age":604800}
server: cloudflare
cf-ray: 8bb977593d529c65-SIN
alt-svc: h3=":443"; ma=86400
```

7. Test Access Restrictions Based on Work Hours

Ensure that Squid enforces the access restrictions during specified work hours.

```
wongpeikey@ubuntu:~$ date
Sat 31 Aug 2024 03:26:44 AM +08
wongpeikey@ubuntu:~$ curl -v -x http://192.168.68.142:3128 http://www.google.com
* Trying 192.168.68.142:3128...
* TCP_NODELAY set
* Connected to 192.168.68.142 (192.168.68.142) port 3128 (#0)
> GET http://www.google.com/ HTTP/1.1
> Host: www.google.com
> User-Agent: curl/7.68.0
> Accept: */*
> Proxy-Connection: Keep-Alive
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 407 Proxy Authentication Required
< Server: squid/4.10
< Mime-Version: 1.0
```

Workhour between 10AM to 7PM can't connect

```
wongpeikey@ubuntu:~$ date
Fri 30 Aug 2024 03:25:41 PM EDT
wongpeikey@ubuntu:~$ curl -v -x http://192.168.68.142:3128 http://www.google.com
* Trying 192.168.68.142:3128...
* TCP_NODELAY set
* Connected to 192.168.68.142 (192.168.68.142) port 3128 (#0)
> GET http://www.google.com/ HTTP/1.1
> Host: www.google.com
> User-Agent: curl/7.68.0
> Accept: */*
> Proxy-Connection: Keep-Alive
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 403 Forbidden
< Server: squid/4.10
< Mime-Version: 1.0
```

8. Test Allowed Ports

Using `curl -x http://<proxy-ip>:3128 http://www.example.com` allows HTTP access through the proxy, while `curl -x http://<proxy-ip>:3128 http://smtp.example.com:25` is denied because port 25 is not listed as a "Safe_port" in the configuration. This setup ensures that only traffic on permitted ports is allowed through the proxy.

```
wongpei@ubuntu:~$ curl -x http://192.168.68.142:3128 http://www.example.com
<!doctype html>
<html>
<head>
  <title>Example Domain</title>

  <meta charset="utf-8" />
  <meta http-equiv="Content-type" content="text/html; charset=utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <style type="text/css">
    body {
      background-color: #f0f0f2;
      margin: 0;
      padding: 0;
      font-family: -apple-system, system-ui, BlinkMacSystemFont, "Segoe UI", "Open Sans", "Helvetica Neue", Helvetica, Arial, sans-serif;
```

```
wongpei@ubuntu:~$ curl -x http://192.168.68.142:3128 http://smtp.example.com:25
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html><head>
<meta type="copyright" content="Copyright (C) 1996-2019 The Squid Software Foundation and contributors">
<meta http-equiv="Content-Type" content="text/html; charset=utf-8">
<title>ERROR: The requested URL could not be retrieved</title>
<style type="text/css">
/*
 * Copyright (C) 1996-2020 The Squid Software Foundation and contributors
 *
 * Squid software is distributed under GPLv2+ license and includes
 * contributions from numerous individuals and organizations.
 * Please see the COPYING and CONTRIBUTORS files for details.
 */
/*
```

2. NGINX

2.1 NGINX Installation

Step 1: Install NGINX on Ubuntu 20.04.6 LTS

1. Update the apt package repository lists.

```
tanyifei@ubuntu:~$ sudo apt update
[sudo] password for tanyifei:
Hit:1 http://archive.ubuntu.com/ubuntu focal InRelease
Hit:2 http://archive.ubuntu.com/ubuntu focal-updates InRelease
Hit:3 http://archive.ubuntu.com/ubuntu focal-backports InRelease
Hit:4 http://archive.ubuntu.com/ubuntu focal-security InRelease
Reading package lists... Done
Building dependency tree
Reading state information... Done
53 packages can be upgraded. Run 'apt list --upgradable' to see them.
tanyifei@ubuntu:~$
```

2. Install NGINX.

```
tanyifei@ubuntu:~$ sudo apt install nginx -y
```

Step 2: Start NGINX and Configure to Launch on Reboot

1. Start the NGINX service.
2. Enable the service to configure NGINX to launch on reboot automatically.

```
tanyifei@ubuntu:~$ sudo systemctl start nginx
tanyifei@ubuntu:~$ sudo systemctl enable nginx
Synchronizing state of nginx.service with SysV service script with /lib/systemd/systemd-sysv-install
.
Executing: /lib/systemd/systemd-sysv-install enable nginx
tanyifei@ubuntu:~$ _
```

Step 3: Verify Nginx Installation

1. After installation, verify that NGINX has been installed correctly by checking its version.

```
tanyifei@ubuntu:~$ nginx -v
nginx version: nginx/1.18.0 (Ubuntu)
tanyifei@ubuntu:~$
```

2. Check if NGINX is running.

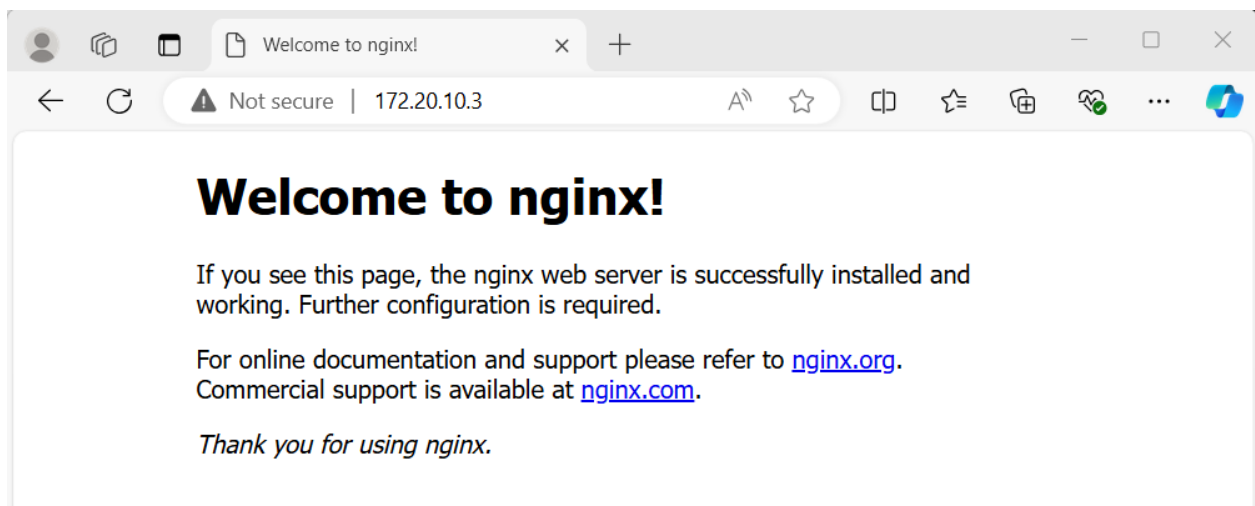
```
tanyifei@ubuntu:~$ sudo service nginx status
• nginx.service - A high performance web server and a reverse proxy server
  Loaded: loaded (/lib/systemd/system/nginx.service; enabled; vendor preset: enabled)
  Active: active (running) since Thu 2024-09-05 12:43:10 UTC; 36s ago
    Docs: man:nginx(8)
  Main PID: 2621 (nginx)
    Tasks: 2 (limit: 2256)
   Memory: 5.1M
   CGroup: /system.slice/nginx.service
           └─2621 nginx: master process /usr/sbin/nginx -g daemon on; master_process on;
             └─2622 nginx: worker process

Sep 05 12:43:10 ubuntu systemd[1]: Starting A high performance web server and a reverse proxy server: nginx.
Sep 05 12:43:10 ubuntu systemd[1]: Started A high performance web server and a reverse proxy server: nginx.
lines 1-13/13 (END)
```

3. Identify the IP address of Ubuntu server.

```
tanyifei@ubuntu:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:14:7b:c0f brd ff:ff:ff:ff:ff:ff
    inet 172.20.10.3/28 brd 172.20.10.15 scope global dynamic enp0s3
        valid_lft 86115sec preferred_lft 86115sec
    inet6 2402:1980:8508:4ebc:a00:27ff:fe47:bc0f/64 scope global mngtmpaddr noprefixroute
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe47:bc0f/64 scope link
        valid_lft forever preferred_lft forever
tanyifei@ubuntu:~$
```

4. Test the NGINX setup by accessing the server's IP address in a browser. It should display the NGINX default welcome page.



2.2 NGINX Configuration

Configuration 1: Set Firewall Rules

1. Allow traffic on port 80.

```
tanyifei@ubuntu:/$ sudo ufw app list
Available applications:
  Nginx Full
  Nginx HTTP
  Nginx HTTPS
  OpenSSH
tanyifei@ubuntu:/$ sudo ufw allow 'Nginx HTTP'
Rules updated
Rules updated (v6)
```

2. Enable the firewall and verify the change. The output will indicate which HTTP traffic is allowed.

```
tanyifei@ubuntu:/$ sudo ufw status
Status: inactive
tanyifei@ubuntu:/$ sudo ufw enable
Firewall is active and enabled on system startup
tanyifei@ubuntu:/$ sudo ufw status
Status: active

To Action From
--
Nginx HTTP ALLOW Anywhere
Nginx HTTP (v6) ALLOW Anywhere (v6)
```

Configuration 2: Create a Website on NGINX Server

Step 1: Set Up a Custom HTML Page

1. Create a directory from the website content.
2. Navigate to the newly created directory.
3. Create an index.html file with a simple HTML page.

```
tanyifei@ubuntu:~$ cd /var/www
tanyifei@ubuntu:/var/www$ sudo mkdir uccn3004
tanyifei@ubuntu:/var/www$ cd uccn3004
tanyifei@ubuntu:/var/www/uccn3004$ sudo "${EDITOR:-vi}" index.html
```

HTML content:

```
<html>
  <head>
    <meta charset='utf-8'>
    <title>Hello, Nginx!</title>
  </head>
  <body>
    <h1>Welcome to the page!</h1>
    <p>We have just configured our Nginx web server on Ubuntu server!</p>
  </body>
</html>
```

Step 2: Configure a Virtual Host (Server Block) on NGINX Server

1. Create a virtual host configuration file in /etc/nginx/sites-available/.

```
tanyifei@ubuntu:/etc/nginx/sites-available$ sudo "${EDITOR:-vi}" uccn3004
```

Configuration file content:

```
server {
    listen 8080;
    listen [::]:8080;

    server_name 172.20.10.3;

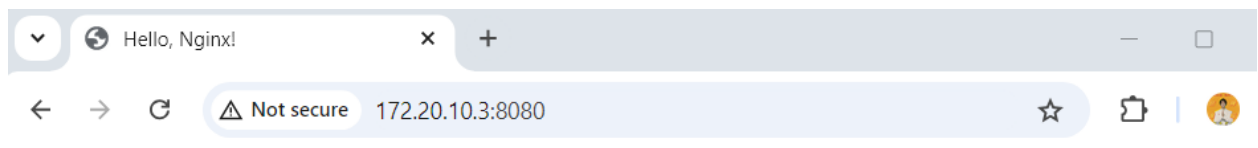
    root /var/www/uccn3004;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

2. Create a symbolic link to the sites-enabled directory.
3. Test the NGINX configuration for syntax errors.
4. Restart NGINX to apply the changes.

```
tanyifei@ubuntu:/etc/nginx/sites-available$ sudo ln -s /etc/nginx/sites-available/uccn3004 /etc/nginx/sites-enabled
tanyifei@ubuntu:/etc/nginx/sites-available$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
tanyifei@ubuntu:/etc/nginx/sites-available$ sudo systemctl restart nginx
tanyifei@ubuntu:/etc/nginx/sites-available$ _
```

5. Browse the ip address to see the newly configured file.



Welcome to the page!

We have just configured our Nginx web server on Ubuntu server!

Configuration 3: Enable Gzip Compression

Gzip compression minimizes file sizes to accelerate network transfers. It is especially useful for enhancing website performance by reducing the size of HTML, CSS, and JavaScript files, which leads to faster data transmission, particularly for users with slower connections.

1. Open the Nginx configuration file.

```
tanyifei@ubuntu:/$ sudo nano /etc/nginx/nginx.conf
```

2. In the http block, add the following lines:

```
##
# Gzip Settings
##

gzip on;

gzip_vary on;
gzip_proxied any;
gzip_comp_level 6;
gzip_buffers 16 8k;
gzip_http_version 1.1;
gzip_types text/plain text/css application/json application/javascript text/xml application/xml;

##
# Virtual Host Configs
```

- **gzip on:** Enables Gzip compression.
- **gzip_vary on:** Instructs proxies to store both compressed and uncompressed versions of resources.
- **gzip_types text/plain text/css text/xml text/javascript application/x-javascript application/xml:** Specifies the MIME types that will be compressed.

3. Test the configuration to make sure there are no syntax errors. If test is successful, reload NGINX configuration.

```
tanyifei@ubuntu:/$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
tanyifei@ubuntu:/$ sudo systemctl reload nginx
tanyifei@ubuntu:/$
```

Configuration 4: Add Security Headers

Within the server block, add security headers to improve protection against common web vulnerabilities.

```
GNU nano 4.8 /etc/nginx/sites-available/uccn3004
server{
    listen 8080;
    listen [::]:8080;
#    listen 443 ssl;

    server_name 172.20.10.3;

    root /var/www/uccn3004;
    index index.html;

    add_header Strict-Transport-Security "max_age=31536000; includeSubDomains" always;
    add_header Referrer-Policy "no-referrer-when-downgrade";
    add_header X-Frame-Options "SAMEORIGIN";
    add_header X-Content-Type-Options "nosniff";
    add_header X-XSS-Protection "1; mode=block";
    add_header Content-Security-Policy "default-src 'self'";

    location / {
        try_files $uri $uri/ =404;
    }
}
```

- **Strict-Transport-Security:** Forces browsers to use HTTPS for future requests (use only if HTTPS is enabled).
- **Referrer-Policy:** Controls the amount of referrer information sent when navigating from your website.
- **X-Frame-Options :** Prevents the site from being embedded in an iframe (protects against clickjacking).
- **X-Content-Type-Options nosniff:** Prevents browsers from interpreting files as a different MIME type.
- **X-XSS-Protection:** Enables cross-site scripting (XSS) protection in browsers.
- **Content-Security-Policy:** Limits the resources the browser can load (e.g., images, scripts, etc.).

Configuration 5: Optimize Worker Processes and Buffers

Client requests are handled by NGINX through worker processes. The total performance capacity is determined by the number of workers and the connections each worker can handle. Enhancing these settings can significantly boost server performance.

1. Open the Nginx configuration file.

```
tanyifei@ubuntu:/$ sudo nano /etc/nginx/nginx.conf_
```

2. In the events block, adjust worker processes.

```
GNU nano 4.8 /etc/nginx/nginx.conf Modified
user www-data;
worker_processes auto;
pid /run/nginx.pid;
include /etc/nginx/modules-enabled/*.conf;

events {
    worker_connections 1024;
    # multi_accept on;
}
```

Configuration 6: Change the Size of the Buffers

Buffer sizes play a crucial role in optimizing NGINX performance. If buffers are too small, NGINX will constantly write to temporary files, causing heavy disk I/O operations. Adjusting buffer sizes can prevent this from occurring.

```
# change the size of the buffers
client_body_buffer_size 10k;
client_header_buffer_size 1k;
client_max_body_size 8m;
large_client_header_buffers 4 4k;
```

- **client_body_buffer_size:** Defines the buffer size for reading client request bodies. If the request exceeds the buffer size, it is temporarily written to a file.
- **client_header_buffer_size:** Specifies the buffer size for client request headers.
- **client_max_body_size:** Limits the maximum size of a client's request body. Requests exceeding this size result in a 413 Request Entity Too Large error.
- **large_client_header_buffers:** Sets the maximum number and size of buffers for large headers. If the request exceeds these buffers, a 414 Request-URI Too Large error is returned.

Configuration 7: Reducing Timeouts

Timeouts can significantly improve NGINX's performance. By maintaining keepalive connections, it reduces CPU and network overhead associated with opening and closing connections.

```
# reducing timeouts
client_body_timeout 12;
client_header_timeout 12;
send_timeout 10;_
```

- **client_header_timeout:** Sets a limit on how long the client request header can be read. If the client doesn't transmit the entire header within this time, a 408 Request Time-out error occurs.
- **client_body_timeout:** Specifies the time allowed to read the client request body between successive reads. If the client doesn't send data within this period, a 408 Request Time-out error is returned.
- **send_timeout:** Sets how long the server will wait for a client to receive data before terminating the connection.

Configuration 8: Rate Limiting

```
http {
    limit_req_zone $binary_remote_addr zone=my_zone:10m rate=5r/s;
    ##
```

```
limit_req zone=my_zone burst=10 nodelay;
try_files $uri $uri/ =404;
```

- **rate=5r/s:** Limits requests to 5 requests per second from a single IP.
- **burst=10:** Allows for a burst of up to 10 requests, after which further requests will be delayed or rejected.
- **nodelay:** requests over the limit will be immediately rejected (without delay).
- Save the file and reload Nginx.

```
tanyifei@ubuntu:/$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
tanyifei@ubuntu:/$ sudo systemctl restart nginx
tanyifei@ubuntu:/$ _
```

Configuration 9: Access Log

Define a custom log format for better tracking of client details.

```
##
log_format custom '$remote_addr - $remote_user [$time_local] "$request" '
                  '$status $body_bytes_sent "$http_referer" '
                  '"$http_user_agent" "$http_x_forwarded_for" ';

access_log /var/log/nginx/access.log custom;
error_log /var/log/nginx/error.log warn;
```

Configuration 10: Host Multiple Websites on Nginx Server with Single IP Address

Step 1: Create Additional Website Blocks

1. Create directories for the new websites.
2. Set the correct permissions. Use the \$USER environment variable to assign ownership of the directory to the currently logged-in account, enabling easy creation or modification of content within the directory.

```
tanyifei@ubuntu:/etc/nginx/sites-available$ cd ..
tanyifei@ubuntu:/etc/nginx$ cd ..
tanyifei@ubuntu:/etc$ cd ..
tanyifei@ubuntu:/$ sudo mkdir -p /var/www/tutorial1.local/html
tanyifei@ubuntu:/$ sudo mkdir -p /var/www/tutorial2.local/html
tanyifei@ubuntu:/$ sudo chown -R $USER:$USER /var/www/tutorial1.local/html
tanyifei@ubuntu:/$ sudo chown -R $USER:$USER /var/www/tutorial2.local/html
tanyifei@ubuntu:/$ sudo chmod -R 755 /var/www
tanyifei@ubuntu:/$
```

Step 2: Create index.html files for both websites:

- For tutorial1.local:

```
tanyifei@ubuntu:/$ sudo vim /var/www/tutorial1.local/html/index.html
```

HTML content:

```
<html>
  <head>
    <title>Welcome to Tutorial 1!</title>
  </head>
  <body>
    <h1>Success! The Tutorial1 server block is working!</h1>
  </body>
</html>
```

- For tutorial2.local:

```
tanyifei@ubuntu:/$ sudo vim /var/www/tutorial2_local/html/index.html
```

HTML content:

```
<html>
  <head>
    <title>Welcome to Tutorial 2!</title>
  </head>
  <body>
    <h1>Success! The Tutorial 2 server block is working!</h1>
  </body>
</html>
```

Step 3: Create server block configuration files:

- For tutorial1.local:

```
tanyifei@ubuntu:/$ sudo vim /etc/nginx/sites-available/tutorial1.local
```

Configuration file content:

```
server{
    listen 80;
    listen [::]:80;

    root /var/www/tutorial1.local/html;
    index index.html index.htm;

    server_name tutorial1.local;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

- For tutorial2.local:

```
tanyifei@ubuntu:/$ sudo vim /etc/nginx/sites-available/tutorial2_local
```

Configuration file content:

```
server{
    listen 80;
    listen [::]:80;

    root /var/www/tutorial2.local/html;
    index index.html index.htm;

    server_name tutorial2.local;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

Step 4: Enabling Server Blocks and Restart Nginx

1. Enable server block files by creating symbolic links in the sites-enabled directory, which NGINX reads during startup.

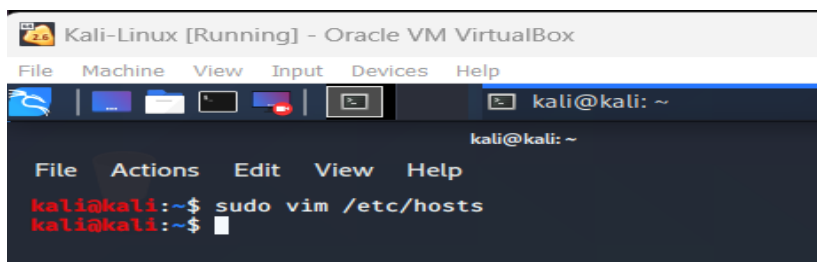
```
tanyifei@ubuntu:/$ sudo ln -s /etc/nginx/sites-available/tutorial1.local /etc/nginx/sites-enabled
tanyifei@ubuntu:/$ sudo ln -s /etc/nginx/sites-available/tutorial2.local /etc/nginx/sites-enabled
```

2. Test the Nginx configuration and restart the service:

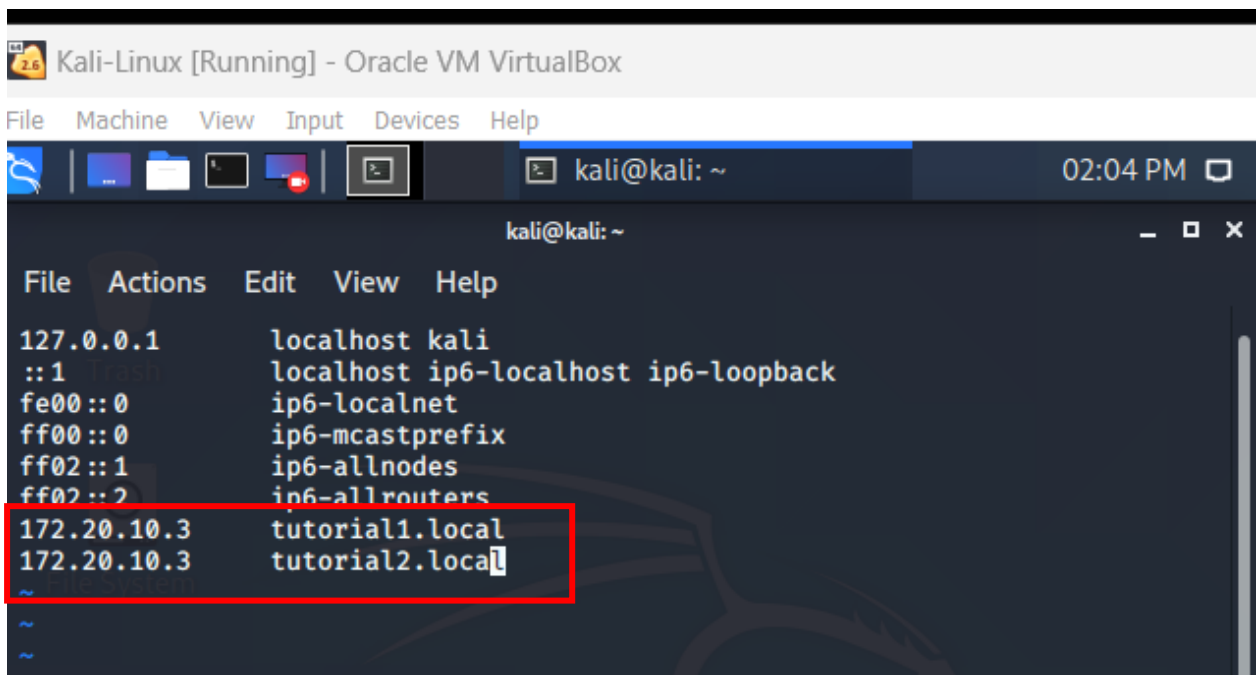
```
tanyifei@ubuntu:/$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
tanyifei@ubuntu:/$ sudo systemctl restart nginx
tanyifei@ubuntu:/$ _
```

Step 5: Update the Local Hosts File (on the Client Machine)

1. Modify /etc/hosts to map the local domains:



Add the following lines:



Configuration 11: Configure NGINX as Reverse Proxy

1. Unlink default configuration file.

NGINX uses a configuration file that is linked by default. To prevent conflicts, unlink it, and create a custom file for setting up the reverse proxy.

```
tanyifei@ubuntu:~$ sudo unlink /etc/nginx/sites-enabled/default
tanyifei@ubuntu:~$ sudo nano /etc/nginx/sites-available/reverse-proxy_
```

2. Configure the NGINX server to function as a reverse proxy.

```
server {
    listen 80;
    server_name 172.20.10.3;

    location / {
        proxy_pass http://172.20.10.3:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

3. Create a symbolic link for the new configuration file and activate it.

```
tanyifei@ubuntu:~$ sudo ln -s /etc/nginx/sites-available/reverse-proxy /etc/nginx/sites-enabled/
tanyifei@ubuntu:~$ _
```

4. Verify the configuration and restart the NGINX server to apply the changes.

```
tanyifei@ubuntu:/$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
tanyifei@ubuntu:/$ sudo systemctl restart nginx
tanyifei@ubuntu:/$ _
```

2.3 System Testing

Verify Gzip Compression

Confirm that Gzip compression is active on the NGINX server. This should minimize the size of data sent to clients, improving website or application performance.

```
tanyifei@ubuntu:/$ curl -I -H 'Accept-Encoding: gzip' http://172.20.10.3:81
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Date: Thu, 05 Sep 2024 18:33:59 GMT
Content-Type: text/html
Last-Modified: Thu, 05 Sep 2024 18:27:07 GMT
Connection: keep-alive
Vary: Accept-Encoding
ETag: W/"66d9f7fb-e5"
Content-Encoding: gzip
```

Verify Security Headers

Check Security Headers. Use the following command to check if the security headers are correctly applied.

```
tanyifei@ubuntu:/$ sudo systemctl restart nginx
tanyifei@ubuntu:/$ curl -I http://172.20.10.3:8080/
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Date: Fri, 06 Sep 2024 09:22:39 GMT
Content-Type: text/html
Content-Length: 225
Last-Modified: Fri, 06 Sep 2024 04:17:30 GMT
Connection: keep-alive
Vary: Accept-Encoding
ETag: "66da825a-e1"
Strict-Transport-Security: max_age=31536000; includeSubDomains
Referrer-Policy: no-referrer-when-downgrade
X-Frame-Options: SAMEORIGIN
X-Content-Type-Options: nosniff
X-XSS-Protection: 1; mode=block
Content-Security-Policy: default-src 'self';
Accept-Ranges: bytes
```

Test Buffer Size

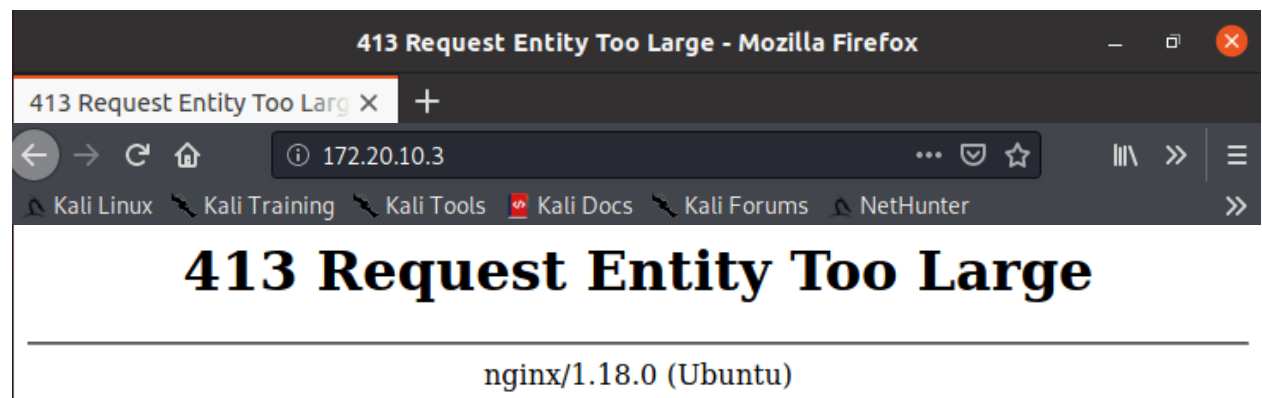
1. To create a file that exceeds the buffer size of 10 KB, we can use the dd command to generate a large test file. For example, here we will create a 20 KB file.

```
kali@kali:~$ dd if=/dev/zero of=largefile.txt bs=1k count=20
20+0 records in
20+0 records out
20480 bytes (20 kB, 20 KiB) copied, 0.000750042 s, 27.3 MB/s
```

2. Upload the large file using curl.

```
kali@kali:~$ curl -X POST --data-binary @largefile.txt http://172.20.10.3/upload
```

Failure: If the file size exceeds the buffer size (client_body_buffer_size), NGINX will throw a 413 Request Entity Too Large error.



Verify Rate Limiting

To test rate limiting, we need to send many requests in a short time and see if NGINX blocks requests that exceed the rate limit. Install Apache Benchmark (ab) to test how many requests per second server can handle. *ab -n 50 -c 10 http://172.20.10.3/*

- -n 50 sends 50 total requests.
- -c 10 sends 10 concurrent requests at once.

```
kali@kali:~$ ab -n 50 -c 10 http://172.20.10.3/
This is ApacheBench, Version 2.3 <$Revision: 1874286 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking 172.20.10.3 (be patient).....done

Server Software:      nginx/1.18.0
Server Hostname:      172.20.10.3
Server Port:          80

Document Path:        /
Document Length:      612 bytes

Concurrency Level:    10
Time taken for tests:  0.015 seconds
Complete requests:    50
Failed requests:      39
  (Connect: 0, Receive: 0, Length: 39, Exceptions: 0)
Non-2xx responses:    39
Total transferred:    24993 bytes
HTML transferred:     14766 bytes
Requests per second:  3255.21 [#/sec] (mean)
Time per request:     3.072 [ms] (mean)
Time per request:     0.307 [ms] (mean, across all concurrent requests)
Transfer rate:        1589.01 [Kbytes/sec] received
```

```
Connection Times (ms)
      min     mean[+/-sd] median   max
Connect:    0      0   0.4      0      3
Processing:  1      2   0.9      2      4
Waiting:    1      2   0.8      2      4
Total:      1      2   0.8      2      4

Percentage of the requests served within a certain time (ms)
 50%      2
 66%      2
 75%      3
 80%      3
 90%      4
 95%      4
 98%      4
 99%      4
100%      4 (longest request)
kali@kali:~$
```

Verify Access Log

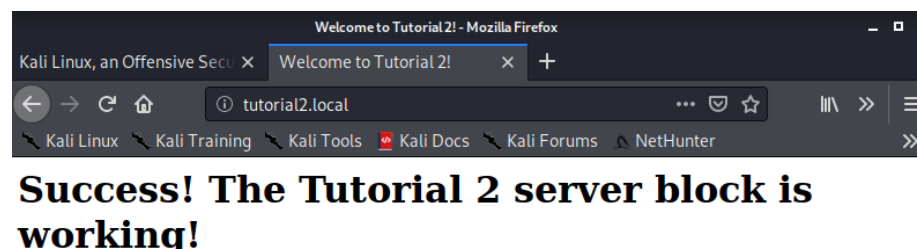
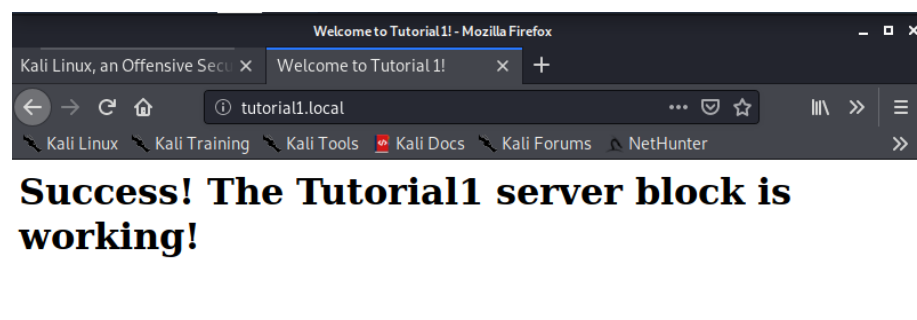
After NGINX is running, we can check the access log for incoming requests. This will display logs in real-time as new requests come in.

```
tanyifei@ubuntu:/$ sudo tail -f /var/log/nginx/access.log
172.20.10.2 - - [06/Sep/2024:04:12:47 +0000] "GET / HTTP/1.1" 200 396 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/128.0.0.0 Safari/537.36 Edg/128.0.0.0"
172.20.10.2 - - [06/Sep/2024:04:12:47 +0000] "GET /favicon.ico HTTP/1.1" 404 197 "http://172.20.10.3/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/128.0.0.0 Safari/537.36 Edg/128.0.0.0"
172.20.10.2 - - [06/Sep/2024:04:21:20 +0000] "GET / HTTP/1.1" 304 0 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/128.0.0.0 Safari/537.36 Edg/128.0.0.0"
172.20.10.2 - - [06/Sep/2024:04:27:05 +0000] "GET / HTTP/1.1" 200 192 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/128.0.0.0 Safari/537.36 Edg/128.0.0.0"
172.20.10.2 - - [06/Sep/2024:04:27:06 +0000] "GET /favicon.ico HTTP/1.1" 404 197 "http://172.20.10.3:8080/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/128.0.0.0 Safari/537.36 Edg/128.0.0.0"
172.20.10.2 - - [06/Sep/2024:04:37:05 +0000] "GET / HTTP/1.1" 200 396 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/128.0.0.0 Safari/537.36"
172.20.10.3 - - [06/Sep/2024:04:44:16 +0000] "HEAD / HTTP/1.1" 200 0 "-" "curl/7.68.0"
172.20.10.3 - - [06/Sep/2024:04:49:50 +0000] "GET / HTTP/1.1" 200 612 "-" "curl/7.68.0"
172.20.10.3 - - [06/Sep/2024:04:51:17 +0000] "GET / HTTP/1.1" 200 225 "-" "curl/7.68.0"
```

Test the Setup of Host Multiple Websites on NGINX Server with Single IP Address

Open your browser and visit:

- <http://tutorial1.local>
- <http://tutorial2.local>



Test Reverse Proxy

1. Create a directory to hold a test page.

```
tanyifei@ubuntu:~$ mkdir ~/example_backend && cd ~/example_backend
tanyifei@ubuntu:~/example_backend$ _
```

2. Create a new file to act as a landing page.

```
tanyifei@ubuntu:~/example_backend$ nano index.html_
```

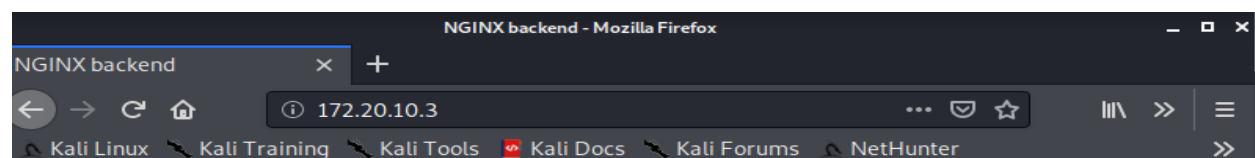
3. Add a basic HTML page.

```
GNU nano 4.8 index.html Modified
html>
head>
    <title>NGINX backend</title>
/head>
body>
    <h1>It works!</h1>
/body>
/html>_
```

4. Start the backend server from the same directory. This will serve the index.html page.

```
tanyifei@ubuntu:~/example_backend$ python3 -m http.server 8000 &
[1] 4865
tanyifei@ubuntu:~/example_backend$ Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
curl 172.20.10.3
172.20.10.3 - - [07/Sep/2024 05:49:53] "GET / HTTP/1.0" 200 -
<html>
<head>
    <title>NGINX backend</title>
</head>
<body>
    <h1>It works!</h1>
</body>
</html>
tanyifei@ubuntu:~/example_backend$
```

5. Access the page in a web browser. The reverse proxy will intercept the client request and forwards it to the backend server, which responds with the HTML page. This verifies that the NGINX reverse proxy is configured correctly.



It works!

3. HAProxy

HAProxy is an open-source HTTP load balancer and reverse proxy system that generally offers high availability to online applications, ensuring maximum uptime. This is a very high-performance application, accelerating web programs by distributing traffic across multiple endpoints. This means that no web server will ever face overload with incoming HTTP requests, as the demand will be equally divided among numerous nodes.

3.1 HAProxy Installation

Step 1: Install HAProxy on Ubuntu 20.04

The command **sudo apt update** is used to refresh the package list and look for available updates.

```
karyeng@ubuntu:~$ sudo apt update
[sudo] password for karyeng:
Hit:1 http://my.archive.ubuntu.com/ubuntu focal InRelease
Get:2 http://my.archive.ubuntu.com/ubuntu focal-updates InRelease [128 kB]
Hit:3 http://my.archive.ubuntu.com/ubuntu focal-backports InRelease
Hit:4 http://my.archive.ubuntu.com/ubuntu focal-security InRelease
Get:5 http://my.archive.ubuntu.com/ubuntu focal-updates/main amd64 Packages [3,533 kB]
Hit:6 http://ppa.launchpad.net/vbernat/haproxy-2.9/ubuntu focal InRelease
Get:7 http://my.archive.ubuntu.com/ubuntu focal-updates/universe amd64 Packages [1,220 kB]
Fetched 4,881 kB in 2s (1,962 kB/s)
Reading package lists... Done
Building dependency tree
Reading state information... Done
60 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

Command **sudo add-apt-repository ppa:vbernat/haproxy-2.9 -y** is used to install a PPA (Personal Package Archive) for HAProxy 2.9 on the Ubuntu system. This allows the user to install the most recent version of HAProxy, which is not included in the usual Ubuntu repository.

```
karyeng@ubuntu:~$ sudo add-apt-repository ppa:vbernat/haproxy-2.9 -y
Hit:1 http://my.archive.ubuntu.com/ubuntu focal InRelease
Hit:2 http://my.archive.ubuntu.com/ubuntu focal-updates InRelease
Hit:3 http://my.archive.ubuntu.com/ubuntu focal-backports InRelease
Hit:4 http://my.archive.ubuntu.com/ubuntu focal-security InRelease
Hit:5 http://ppa.launchpad.net/vbernat/haproxy-2.9/ubuntu focal InRelease
Reading package lists... Done
```

The command **sudo apt install -y haproxy=2.9.*** is used to install the most recent version of HAProxy (2.9.9-1ppa1 focused) on the Ubuntu system.

```
karyeng@ubuntu:~$ sudo apt install -y haproxy=2.9.*
Reading package lists... Done
Building dependency tree
Reading state information... Done
haproxy is already the newest version (2.9.9-1ppa1~focal).
Selected version '2.9.9-1ppa1~focal' (HAProxy 2.9:20.04/focal [amd64]) for 'haproxy'
0 upgraded, 0 newly installed, 0 to remove and 60 not upgraded.
```

The command **haproxy -v** is used to verify the version of HAProxy installed on the system.

```
karyeng@ubuntu:~$ haproxy -v
HAProxy version 2.9.9-1ppa1~focal 2024/06/14 - https://haproxy.org/
Status: stable branch - will stop receiving fixes around Q1 2025.
Known bugs: http://www.haproxy.org/bugs/bugs-2.9.9.html
Running on: Linux 5.4.0-193-generic #213-Ubuntu SMP Fri Aug 2 19:14:16 UTC 2024 x86_64
```

The command **sudo systemctl status haproxy** is used to determine the status of the HAProxy service.

```
karyeng@ubuntu:~$ sudo systemctl status haproxy
• haproxy.service - HAProxy Load Balancer
  Loaded: loaded (/lib/systemd/system/haproxy.service; enabled; vendor preset: enabled)
  Active: active (running) since Sat 2024-08-31 10:24:13 UTC; 4min 8s ago
    Docs: man:haproxy(1)
           file:/usr/share/doc/haproxy/configuration.txt.gz
  Main PID: 681 (haproxy)
    Status: "Ready."
     Tasks: 2 (limit: 2256)
    Memory: 47.1M
    CGroup: /system.slice/haproxy.service
            └─681 /usr/sbin/haproxy -Ws -f /etc/haproxy/haproxy.cfg -p /run/haproxy.pid -S /run/hap
              782 /usr/sbin/haproxy -Ws -f /etc/haproxy/haproxy.cfg -p /run/haproxy.pid -S /run/hap

Aug 31 10:24:12 ubuntu systemd[1]: Starting HAProxy Load Balancer...
Aug 31 10:24:13 ubuntu haproxy[681]: [NOTICE] (681) : New worker (782) forked
Aug 31 10:24:13 ubuntu systemd[1]: Started HAProxy Load Balancer.
Aug 31 10:24:13 ubuntu haproxy[681]: [NOTICE] (681) : Loading success.
lines 1-17/17 (END)
```

The command **sudo systemctl enable haproxy** is used to enable the HAProxy service to start automatically when the system boots.

```
karyeng@ubuntu:~$ sudo systemctl enable haproxy
Synchronizing state of haproxy.service with SysV service script with /lib/systemd/systemd-sysv-instal
11.
Executing: /lib/systemd/systemd-sysv-install enable haproxy
```

The command **ifconfig** displays the network interface setup for the Ubuntu system. The Ubuntu server's IP address is **192.168.0.118**.

```
karyeng@ubuntu:~$ ifconfig
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.0.118 netmask 255.255.255.0 broadcast 192.168.0.255
    inet6 fe80::a00:27ff:fe0c:23ce prefixlen 64 scopeid 0x20<link>
    inet6 2001:e68:543a:d39b:a00:27ff:fe0c:23ce prefixlen 64 scopeid 0x0<global>
    ether 08:00:27:0c:23:ce txqueuelen 1000 (Ethernet)
    RX packets 21042 bytes 30479491 (30.4 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 4785 bytes 388856 (388.8 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 206 bytes 18604 (18.6 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 206 bytes 18604 (18.6 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

3.2 HAProxy Configuration

Step 2: Configure HAProxy

The command `sudo cp /etc/haproxy/haproxy.cfg /etc/haproxy/haproxy.cfg.bk` is used to generate a backup of the HAProxy configuration file. This is a recommended practice before making any changes to the configuration file since it allows us to revert to the original configuration if needed.

```
karyeng@ubuntu:~$ sudo cp /etc/haproxy/haproxy.cfg /etc/haproxy/haproxy.cfg.bk
```

The command `sudo nano /etc/haproxy/haproxy.cfg` opens the HAProxy configuration file in the Nano text editor. This allows the user to make modifications to the configuration files.

```
karyeng@ubuntu:~$ sudo nano /etc/haproxy/haproxy.cfg_
```

Insert the following lines into the HAProxy configuration file.

```
frontend http
    bind *:80
    stats uri /haproxy?stats
    default_backend web-servers
    stick-table type ip size 1m expire 10s store http_req_rate(15s)
    http-request track-sc0 src
    http-request deny if { sc_http_req_rate(0) gt 15 }

backend web-servers
    cookie SERVER_USED insert indirect nocache
    balance roundrobin
    server web1 192.168.0.134:80 check cookie web1
    server web2 192.168.0.133:80 check cookie web2

listen stats
    bind *:8080
    stats enable
    stats uri /
    stats refresh 5s
    stats realm Haproxy\ Statistics
    stats auth karyeng:ubuntu
```

Frontend Section

- **frontend http:** Determines the name of the frontend section.
- **bind *:80:** The frontend will listen for incoming connections on port 80 of all network interfaces.
- **stats uri /haproxy?stats:** Enables frontend statistics and specifies the URL through which they may be accessed.
- **default_backend web-servers:** Configures the default backend section to handle incoming requests.

- **stick-table type ip size 1m expire 10s store http_req_rate(15s):** Creates a stick table to hold client session data. The table is of type ip, has a size of 1 MB, expires in 10 seconds, and contains the HTTP request rate during a 15-second period.
- **http-request track-sco src:** Tracks the source IP address of each client request.
- **http-request deny if { sc_http_req_rate(0) gt 15 }:** Denies client requests that exceed the rate restriction of 15 per second.

Backend Section

- **backend web-servers:** Defines the name of the backend section.
- **cookie SERVER_USED insert indirect nocache:** The cookie SERVER_USED enables session persistence.
- **balance roundrobin:** Specifies the round robin load balancing algorithm.
- **server web1 192.168.0.134:80 check cookie web1:** Determines the first web server in the backend pool.
- **server web2 192.168.0.133:80 check cookie web2:** Determines the second web server in the backend pool.

Listen Stats Section

- **listen stats:** Defines a listener for statistics.
- **bind *:8080:** Specifies that the listener will bind to port 8080.
- **stats enable:** Enables statistics.
- **stats uri /:** Sets the URI for accessing statistics.
- **stats refresh 5s:** Sets the statistics refresh interval to 5 seconds.
- **stats realm Haproxy Statistics:** Sets the realm for HTTP authentication.
- **stats auth karyeng:ubuntu:** Sets the username and password for HTTP authentication.

Save all the changes and exit the configuration file. Reload the updated settings by restarting the haproxy service.

```
karyeng@ubuntu:~$ sudo systemctl restart haproxy
```

Next, edit the `/etc/hosts` file.

```
karyeng@ubuntu:~$ sudo nano /etc/hosts_
```

Define the hostnames and IP addresses for the haproxy main server and webservers.

```
127.0.0.1 localhost
127.0.1.1 ubuntu

192.168.0.118 haproxy
192.168.0.134 web1
192.168.0.133 web2

# The following lines are desirable for IPv6 capable hosts
::1      ip6-localhost ip6-loopback
fe00::0  ip6-localnet
ff00::0  ip6-mcastprefix
ff02::1  ip6-allnodes
ff02::2  ip6-allrouters
```

Step 3: Configure Web Servers for HAProxy

I installed Apache2 on 2 virtual machines.

Webserver1

The command `ifconfig` displays the network interface configuration for the first virtual machine. The webserver1's IP address is **192.168.0.134**.

```
kali@kali:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group d
efault qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state
UP group default qlen 1000
    link/ether 08:00:27:4e:de:40 brd ff:ff:ff:ff:ff:ff
    inet 192.168.0.134/24 brd 192.168.0.255 scope global dynamic noprefixro
ute eth0
        valid_lft 86346sec preferred_lft 86346sec
    inet6 2001:e68:543a:c00b:1ba7:963d:fe03:1532/64 scope global dynamic no
prefixroute
        valid_lft 86348sec preferred_lft 1748sec
    inet6 fe80::5c72:1da:fcbe:1f79/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

On a Kali Linux system, use the command `sudo apt update` to refresh the package lists and search for available updates.

```
kali@kali:~$ sudo apt update
Get:1 http://kali.download/kali kali-rolling InRelease [41.5 kB]
Err:1 http://kali.download/kali kali-rolling InRelease
  The following signatures were invalid: EXPKEYSIG ED444FF07D8D0BF6 Kali Li
  nux Repository <devel@kali.org>
Reading package lists... Done
Building dependency tree
Reading state information... Done
30 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

The command **sudo apt install -y apache2** is used to install Apache on a Kali Linux system.

```
kali@kali:~$ sudo apt install -y apache2
Reading package lists... Done
Building dependency tree
Reading state information... Done
apache2 is already the newest version (2.4.43-1).
apache2 set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 30 not upgraded.
```

The command **sudo systemctl status apache2** is used to determine the status of the Apache2 service. The output shows that Apache2 is operating, active, and loaded, and that the service was successfully launched on August 31, 2024.

```
kali@kali:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/lib/systemd/system/apache2.service; enabled; vendor >
   Active: active (running) since Sat 2024-08-31 10:46:35 UTC; 15s ago
     Docs: https://httpd.apache.org/docs/2.4/
   Process: 1804 ExecStart=/usr/sbin/apachectl start (code=exited, status>
   Main PID: 1815 (apache2)
     Tasks: 6 (limit: 2093)
    Memory: 20.1M
   CGroup: /system.slice/apache2.service
           └─1815 /usr/sbin/apache2 -k start
             └─1816 /usr/sbin/apache2 -k start
               └─1817 /usr/sbin/apache2 -k start
                 └─1818 /usr/sbin/apache2 -k start
                   └─1819 /usr/sbin/apache2 -k start
                     └─1820 /usr/sbin/apache2 -k start

Aug 31 10:46:34 kali systemd[1]: Starting The Apache HTTP Server ...
Aug 31 10:46:35 kali apachectl[1814]: AH00558: apache2: Could not reliably>
Aug 31 10:46:35 kali systemd[1]: Started The Apache HTTP Server.
lines 1-19/19 (END)
```

The command **sudo systemctl enable apache2** is used to configure the Apache2 service to start automatically when the system boots.

```
kali@kali:~$ sudo systemctl enable apache2
Synchronizing state of apache2.service with SysV service script with /lib/s
ysvinit/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable apache2
Created symlink /etc/systemd/system/multi-user.target.wants/apache2.service
→ /lib/systemd/system/apache2.service.
```

Configure webserver1

The command **sudo su** is executed to switch to the root user in a Kali Linux terminal.

```
kali@kali:~$ sudo su
root@kali:/home/kali#
```

The command **echo "<H1>Welcome to webserver1: 192.168.0.134 </H1>" > /var/www/html/index.html** generates a new HTML file named index.html in the /var/www/html directory, which is the default document root for Apache2 web server1.

```
root@kali:/home/kali# echo "<H1>Welcome to webserver1 : 192.168.0.134 </H1>" > /var/www/html/index.html
```

The command **sudo nano /etc/hosts** is executed to open the /etc/hosts file in the Nano text editor. This file is used to map hostnames to IP addresses on a Linux system.

```
kali@kali:~$ sudo nano /etc/hosts
```

Add the HAProxy entry.

```
127.0.0.1    localhost kali
::1         localhost ip6-localhost ip6-loopback
fe00::0     ip6-localnet
ff00::0     ip6-mcastprefix
ff02::1     ip6-allnodes
ff02::2     ip6-allrouters
192.168.0.118 haproxy
```

Make sure you can ping the HAProxy server from webserver1.

```
kali@kali:~$ ping haproxy -c 3
PING haproxy (192.168.0.118) 56(84) bytes of data.
64 bytes from haproxy (192.168.0.118): icmp_seq=1 ttl=64 time=0.549 ms
64 bytes from haproxy (192.168.0.118): icmp_seq=2 ttl=64 time=0.820 ms
64 bytes from haproxy (192.168.0.118): icmp_seq=3 ttl=64 time=0.246 ms

--- haproxy ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2056ms
rtt min/avg/max/mdev = 0.246/0.538/0.820/0.234 ms
```

Webserver2

The command **ifconfig** shows the network interface setup for the second virtual machine. The IP address for webserver2 is **192.168.0.133**.

```
lubuntu@lubuntu:~$ ifconfig
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.0.133 netmask 255.255.255.0 broadcast 192.168.0.255
    inet6 2001:e68:543a:d39b:fe38:8ec8:b0d1:851e prefixlen 64 scopeid 0
x0<global>
    inet6 2001:e68:543a:d39b:37ce:9817:feea:77ad prefixlen 64 scopeid 0
x0<global>
    inet6 fe80::c285:3c1d:3093:3395 prefixlen 64 scopeid 0x20<link>
    ether 08:00:27:77:95:1b txqueuelen 1000 (Ethernet)
    RX packets 30750 bytes 25421399 (25.4 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 12059 bytes 1437124 (1.4 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 248 bytes 26278 (26.2 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 248 bytes 26278 (26.2 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

The command **sudo apt update** is used to refresh package lists and search for available updates on a Lubuntu system.

```
lubuntu@lubuntu:~$ sudo apt update
```

The command **sudo apt install -y apache2** is used to install Apache on a Lubuntu system.

```
lubuntu@lubuntu:~$ sudo apt install -y apache2
```

The command **sudo systemctl status apache2** is used to determine the status of the Apache2 service. The result shows that Apache2 is operating, active, and loaded, and that the service was successfully launched on August 31, 2024.

```

lubuntu@lubuntu:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/lib/systemd/system/apache2.service; enabled; vendor pr
   Active: active (running) since Sat 2024-08-31 11:01:23 UTC; 1min 16s ago
     Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 2750 (apache2)
    Tasks: 55 (limit: 2186)
   Memory: 4.8M
      CPU: 48ms
   CGroup: /system.slice/apache2.service
           └─2750 /usr/sbin/apache2 -k start
             └─2752 /usr/sbin/apache2 -k start
               └─2753 /usr/sbin/apache2 -k start

Aug 31 11:01:23 lubuntu systemd[1]: Starting The Apache HTTP Server...
Aug 31 11:01:23 lubuntu apachectl[2749]: AH00558: apache2: Could not reliabl
Aug 31 11:01:23 lubuntu systemd[1]: Started The Apache HTTP Server.
lines 1-16/16 (END)

```

The command **sudo systemctl enable apache2** is used to configure the Apache2 service to start automatically when the system boots.

```

lubuntu@lubuntu:~$ sudo systemctl enable apache2
Synchronizing state of apache2.service with SysV service script with /lib/sys
temd/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable apache2

```

Configure webserver2

The command **sudo su** is executed to switch to the root user in a Ubuntu terminal.

```

lubuntu@lubuntu:~$ sudo su
root@lubuntu:/home/lubuntu#

```

The command **echo "<H1>Hello! Welcome to webserver2: 192.168.0.133 </H1>" > /var/www/html/index.html** is being executed to generate a new HTML file called index.html in the /var/www/html directory, which is the default document root for the Apache2 web server.

```

root@lubuntu:/home/lubuntu# echo "<H1>Hello! Welcome to webserver2 : 192.168.
0.133 </H1>" > /var/www/html/index.html

```

The command **sudo nano /etc/hosts** is executed to open the /etc/hosts file in the Nano text editor. This file is used to map hostnames to IP addresses on a Linux system.

```

lubuntu@lubuntu:~$ sudo nano /etc/hosts

```

Add the HAProxy entry.

```
127.0.0.1 localhost
127.0.1.1 lubuntu

192.168.0.118 haproxy

# The following lines are desirable for IPv6 capable hosts
::1          ip6-localhost ip6-loopback
fe00::0      ip6-localnet
ff00::0      ip6-mcastprefix
ff02::1      ip6-allnodes
ff02::2      ip6-allrouters
ff02::3      ip6-allhosts
```

Make sure you can ping the HAProxy server from webserver2.

```
lubuntu@lubuntu:~$ ping haproxy -c 3
PING haproxy (192.168.0.118) 56(84) bytes of data.
64 bytes from haproxy (192.168.0.118): icmp_seq=1 ttl=64 time=4.66 ms
64 bytes from haproxy (192.168.0.118): icmp_seq=2 ttl=64 time=5.45 ms
64 bytes from haproxy (192.168.0.118): icmp_seq=3 ttl=64 time=13.1 ms

--- haproxy ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2072ms
rtt min/avg/max/mdev = 4.657/7.732/13.086/3.799 ms
```

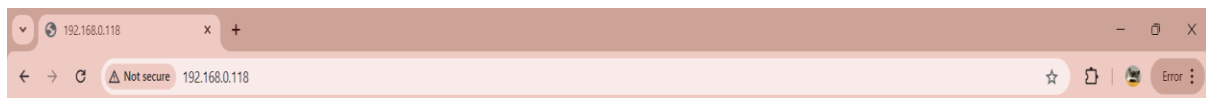
3.3 System Testing

Load Balancing

Load balancing is a technique for distributing incoming network traffic among many servers. This improves performance, reliability, and scalability. To test a load balancer, we can integrate website testing with command-line tools such as curl.

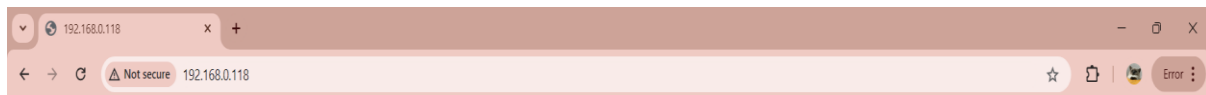
Browse the IP address of the HAProxy server: **http://192.168.0.118**

When we initially start browsing, we view the first web server's webpage.



Welcome to webserver1 : 192.168.0.134

After refreshing, we see the webpage for the second web server.



Hello! Welcome to webserver2 : 192.168.0.133

Try connecting to 192.168.0.118 using the curl command:

```
> User-Agent: curl/7.68.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< date: Sat, 31 Aug 2024 20:05:59 GMT
< server: Apache/2.4.43 (Debian)
< last-modified: Sat, 31 Aug 2024 10:48:52 GMT
< etag: "30-620f8741c1690"
< accept-ranges: bytes
< content-length: 48
< content-type: text/html
<
<H1>Welcome to webserver1 : 192.168.0.134 </H1>
* Connection #0 to host 192.168.0.118 left intact
karyeng@ubuntu:~$ curl -v 192.168.0.118
* Trying 192.168.0.118:80...
* TCP_NODELAY set
* Connected to 192.168.0.118 (192.168.0.118) port 80 (#0)
> GET / HTTP/1.1
> Host: 192.168.0.118
> User-Agent: curl/7.68.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< date: Mon, 02 Sep 2024 03:29:02 GMT
< server: Apache/2.4.52 (Ubuntu)
< last-modified: Sat, 31 Aug 2024 11:20:11 GMT
< etag: "37-620f8e41b0325"
< accept-ranges: bytes
< content-length: 55
< content-type: text/html
<
<H1>Hello! Welcome to webserver2 : 192.168.0.133 </H1>
* Connection #0 to host 192.168.0.118 left intact
```

We can see that HAProxy is spreading traffic across Webservers 1 and 2.

Statistics Page

With the curl command, we can utilize the do-while loop with the curl command below.

```
karyeng@ubuntu:~$ while true; do curl 192.168.0.118; sleep 1; done
```

To access monitoring statistics, navigate to **http://192.168.0.118:8080/stats** and enter the username and password.

The screenshot shows a web browser window with the URL `192.168.0.118:8080/stats`. A sign-in dialog is displayed in the center, asking for a username and password. The username is `karyeng` and the password is masked with dots. Below the dialog, the main statistics page is visible, titled "HAProxy version 2.9.9-1ppa1~focal, released 2024/06/14". The page shows a "Statistics Report for pid 12121" and includes a "General process information" section. The main part of the page displays a table of statistics for the "http" and "stats" sections. The "http" section shows a summary of sessions, bytes, and errors. The "stats" section shows a detailed breakdown of sessions, including a list of active connections and a summary of response times.

Sign in

http://192.168.0.118:8080

Your connection to this site is not private

Username karyeng

Password

Sign in Cancel

HAProxy version 2.9.9-1ppa1~focal, released 2024/06/14

Statistics Report for pid 12121

> General process information

pid = 12121 (process #1, nbproc = 1, nbthread = 1)
uptime = 0d 0h36m59s, warnings = 0
system limits: memmax = unlimited, ulimit = 524287
maxsock = 524287, maxconn = 262144, reached = 0, maxpipes = 0
current conn = 2, current pipes = 0, conn rate = Disc; 1/s rate = 0.000 kbps
Running tasks: 0/17 (0 nice); idle = 100 %

active UP active UP, going down backup UP backup UP, going down
active DOWN, going up active DOWN, going up
active or backup DOWN not checked
active or backup DOWN for maintenance (MAINT)
active or backup SOFT STOPPED for maintenance
Note: "NOLEGITDRAIN" = UP with load-balancing disabled

Display option:
• Scope:
• Hide DOWN servers
• Disable refresh
• Refresh now
• CSV export
• JSON export (schema)

External resources:
• Primary site
• Updates (1/2 S)
• Online manual

http		Session rate		Sessions		Bytes		Denied		Errors		Warnings		Status		Server	
Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Limit	Cur	Max	
Frontend	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
...																	

stats		Session rate		Sessions		Bytes		Denied		Errors		Warnings		Status		Server	
Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Limit	Cur	Max	
Frontend	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
Backend	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
...																	

Cum. sessions: 468
New connections: 491
Reused connections: 1 (0%)
Cum. HTTP requests: 468
- HTTP 1xx responses: 0
- HTTP 2xx responses: 467 (0%)
- HTTP 3xx responses: 2
- HTTP 4xx responses: 7
- other responses: 0
Cache hits: 0 (0%)
Cache lookups: 0
Failed hfr rewrites: 0
Internal errors: 0
Max / Avg over last 1024 success. conn.
- Queue time: 0 / 0 ms
- Connect time: 11 309 / 30 ms
- Response time: 108 / 2 ms
- Total time: 11 310 / 35 ms

Health checking

Health checking is the process of monitoring the status of individual servers within a pool to verify that they are operational and capable of handling incoming requests. This keeps users from being routed to servers that are offline or experiencing problems.

We try manually stopping Webserver2 to check if HAProxy can detect the server offline and redirect traffic.

```
lubuntu@lubuntu:~$ sudo systemctl stop apache2
lubuntu@lubuntu:~$ sudo systemctl status apache2
○ apache2.service - The Apache HTTP Server
   Loaded: loaded (/lib/systemd/system/apache2.service; enabled; vendor pr>
   Active: inactive (dead) since Mon 2024-09-02 03:22:59 UTC; 18s ago
     Docs: https://httpd.apache.org/docs/2.4/
   Process: 3625 ExecStop=/usr/sbin/apachectl graceful-stop (code=exited, s>
   Main PID: 2750 (code=exited, status=0/SUCCESS)
    CPU: 2.137s
```

Perform the **curl** command to check the connection.

```
karyeng@ubuntu:~$ curl -v 192.168.0.118
* Trying 192.168.0.118:80...
* TCP_NODELAY set
* Connected to 192.168.0.118 (192.168.0.118) port 80 (#0)
> GET / HTTP/1.1
> Host: 192.168.0.118
> User-Agent: curl/7.68.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< date: Sat, 31 Aug 2024 20:01:55 GMT
< server: Apache/2.4.43 (Debian)
< last-modified: Sat, 31 Aug 2024 10:48:52 GMT
< etag: "30-620f8741c1690"
< accept-ranges: bytes
< content-length: 48
< content-type: text/html
<
<H1>Welcome to webserver1 : 192.168.0.134 </H1>
* Connection #0 to host 192.168.0.118 left intact
```

The response should now only be provided by Webserver1.

Request Rate Limiting

Request Rate Limiting is a technique for controlling the amount of requests a client may make in a given time period. This helps to avoid overloading a server or API with too many queries, which can degrade performance and availability.

The command **ab -n 20 -c 5 http://192.168.0.118/** is used to run a benchmark test on the web server with the IP address 192.168.0.118. The -n 20 option sends 20 parallel requests, whereas the -c 5 option uses 5 concurrent connections.

```
karyeng@ubuntu:~$ ab -n 20 -c 5 http://192.168.0.118/

Server Hostname: 192.168.0.118
Server Port: 80

Document Path: /
Document Length: 48 bytes

Concurrency Level: 5
Time taken for tests: 0.010 seconds
Complete requests: 20
Failed requests: 5
  (Connect: 0, Receive: 0, Length: 5, Exceptions: 0)
Non-2xx responses: 5
Total transferred: 6670 bytes
HTML transferred: 1190 bytes
Requests per second: 1940.99 [#/sec] (mean)
Time per request: 2.576 [ms] (mean)
Time per request: 0.515 [ms] (mean, across all concurrent requests)
Transfer rate: 632.15 [Kbytes/sec] received
```

The request rate restriction of 15 requests per second, including 15 unsuccessful requests, indicates that request rate limiting is functioning.

Cookies

Cookies are used to ensure session persistence. This implies that all requests from the same client are routed to the same server in the backend pool.

The command **curl -b cookie.txt http://192.168.0.118:80/** is run twice to deliver HTTP requests to the load balancer located at IP address 192.168.0.118.

```
karyeng@ubuntu:~$ curl -b cookie.txt http://192.168.0.118:80/
<H1>Welcome to webserver1 : 192.168.0.134 </H1>
karyeng@ubuntu:~$ curl -b cookie.txt http://192.168.0.118:80/
<H1>Hello! Welcome to webserver2 : 192.168.0.133 </H1>
```

The command **curl -I -b cookie.txt http://192.168.0.118:80/** performs an HTTP request to a web server using a cookie saved in a file and retrieves just the response's HTTP headers.

```
karyeng@ubuntu:~$ curl -I -b cookie.txt http://192.168.0.118:80/
HTTP/1.1 200 OK
date: Sat, 31 Aug 2024 22:36:22 GMT
server: Apache/2.4.43 (Debian)
last-modified: Sat, 31 Aug 2024 10:48:52 GMT
etag: "30-620f8741c1690"
accept-ranges: bytes
content-length: 48
content-type: text/html
set-cookie: SERVER_USED=web1; path=/
cache-control: private

karyeng@ubuntu:~$ curl -I -b cookie.txt http://192.168.0.118:80/
HTTP/1.1 200 OK
date: Mon, 02 Sep 2024 06:00:54 GMT
server: Apache/2.4.52 (Ubuntu)
last-modified: Sat, 31 Aug 2024 11:20:11 GMT
etag: "37-620f8e41b0325"
accept-ranges: bytes
content-length: 55
content-type: text/html
set-cookie: SERVER_USED=web2; path=/
cache-control: private
```

I enabled cookie-based session persistence, which means the load balancer sends an HTTP cookie to the client's browser with the server's unique cookie values (web1 and web2). The client adds the cookie to each future request so that the load balancer can determine which server to utilize. For example, if a client is initially routed to server web1, it will continue to be delivered to that server.